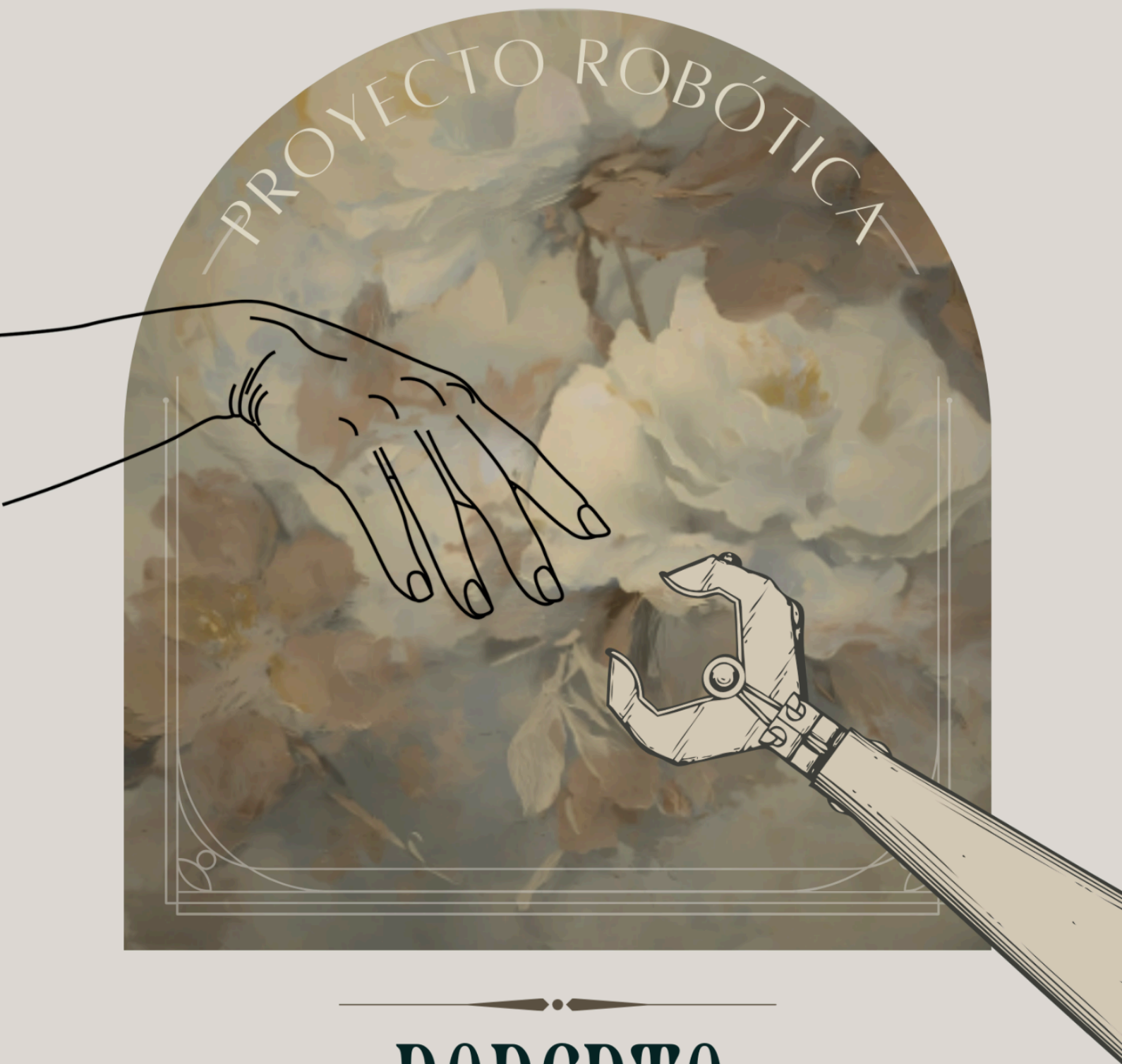


Meryame Ait Boumlik | Christopher Yoris | Maria Algora



ROBERTO

ROBOT PARA EL AEROPUERTO

ÍNDICE

ÍNDICE General	2
FIGURAS	3
1. Introducción	5
2. Contexto del proyecto	6
Stakeholders principales	6
Restricciones operativas	6
Restricciones técnicas	6
Restricciones de negocio	7
Dependencias externas	7
3. Objetivos del proyecto	8
Métricas de éxito	8
4. Definición de requisitos del proyecto	10
Requisitos funcionales	10
Requisitos técnicos	10
Requisitos de usuario	11
Restricciones técnicas	11
Priorización de requisitos	12
Criterios de aceptación	13
5. Diseño	14
6. Implementación	29
6.1 Listado de componentes implementados	30
6.2 Detalle de la implementación de componentes	31
6.2.1 Nodo de detección de obstáculos (roberto_lidar)	31
6.2.2 Simulación y modelado (roberto_mundo)	31
6.2.3 Localización adaptativa (roberto_nav_punto)	31
6.2.4 Navegación por waypoints y objetivos (roberto_nav_ruta)	32
6.2.5 Metapaquete Roberto	32
6.3 Pagina web (Tecnico)	32
6.3.1 Interfaz web de visualización (main.js)	32
6.3.2 Servidor web (server.js)	33
6.3.3 API REST (api.js)	33
6.3.4 Cliente ROS (rosClient.js)	33
6.3.5 Persistencia de datos (logica.js)	33
6.3.6 Nodo de control del robot (simple_follower.py)	33
6.4 Página web (Usuario)	34
6.3.1 Pantalla inicial y selección de idioma	34
6.3.2 Selección de destino	34
6.3.3 Navegación en curso	34
6.3.4 Pantalla de llegada y valoración	34
7. Verificación de las funcionalidades implementadas	35
7.1 Listado de pruebas realizadas	35
7.2 Detalle de las pruebas ejecutadas	36

7.2.1 Navegación autónoma	36
7.2.2 Detección de obstáculos con LiDAR	38
7.2.3 Localización mediante AMCL	39
7.2.4 Conexión	42
7.2.5 Control	43
7.2.6 Estado	44
7.2.7 Login	45
7.2.7 Mapa en Web	47
7.2.8 Cámara	49
7.2.9 Interfaz pasajero	50
7.2.10 Historial	53
7.2.11 Detección de Personas (OpenCV)	55
7.2.13 Detección de gestos (OpenCV)	56
7.3 Conclusión de las pruebas	57
8. Funcionamiento del grupo	58
Actas	58
Burndown	62
Anexo 1. Tabla de historial de versiones	64

ÍNDICE Figuras.....	2
Figura : Arquitectura completa.....	14
Figura : Diseño de paquete de localización.....	15
Figura : Diseño de paquete LiDAR.....	16
Figura : Diseño de paquete navegación.....	17
Figura : Diseño de flujo de visualizacion del mapa.....	18
Figura : Diseño de conexión del Robot.....	19
Figura : Diseño de Control Manual del Robot.....	20
Figura : MoodBoard.....	21
Figura : Pantalla inicial.....	22
Figura : Selección de terminal.....	23
Figura: Cargar maletas.....	24
Figura : En camino.....	25
Figura: Llegado.....	25
Figura: Interfaz del técnico.....	26
Figura: BBDD.....	27
Figura: Estructura de carpetas.....	29
Figuras : Navegación.....	37
Figura : Obstacle detection y LiDAR.....	39
Figuras : Localización.....	42
Figura : Conexión.....	42
Figura : Control.....	43
Figura : Estado.....	44
Figura : Login.....	45

Figuras : Mapa.....	48
Figura : Camara.....	49
Figuras : Interfaz pasajero funcional.....	52
Figuras : Historial.....	53
Figura : Deteccion de Personas OpenCV.....	55
Figura : Deteccion de gestos OpenCV.....	56
Figura : Burndown Sprint 1.....	62
Figura : Burndown Sprint 2.....	63
Figura : Burndown Sprint 3.....	64

1. Introducción

En los últimos años, los aeropuertos se han consolidado como espacios de alta complejidad operativa y tecnológica, donde la circulación constante de pasajeros y equipajes requiere soluciones que optimicen la movilidad y reduzcan el estrés del usuario. Las largas distancias entre las zonas de facturación, seguridad y embarque suelen generar confusión y fatiga, especialmente en viajeros con prisa o con equipaje pesado. En este contexto, la robótica de servicio se presenta como una herramienta clave para mejorar la experiencia del viajero.

Este documento está dirigido a ingenieros, desarrolladores, gestores de proyectos, stakeholders aeroportuarios y equipos de integración en un contexto de despliegue de robots móviles autónomos en aeropuertos, priorizando seguridad operativa, integración con infraestructuras críticas y cumplimiento normativo específico del sector aeronáutico.

Para un robot guía en aeropuertos, las referencias principales son las normas sobre robots de servicio y robots móviles que comparten espacio con peatones, junto con las reglas de seguridad general, protección de datos y accesibilidad. En concreto, **ISO 13482** cubre robots de servicio personal, **ISO 3691-4** regula robots móviles autónomos en entornos mixtos, e **ISO 12100** sirve como base para analizar y reducir riesgos. Como el robot usa **cámara e inteligencia artificial**, también le aplican la normativa de protección de datos y el **Reglamento europeo de IA**. Además, sus pantallas y sistemas de interacción deben cumplir normas de accesibilidad como **EN 301 549** y **WCAG 2.2**. Si integra funciones de seguridad más críticas, puede tomarse como referencia **IEC 61508**. Finalmente, debe ajustarse a los requisitos operativos del aeropuerto, especialmente en circulación, emergencias y ciberseguridad.

2. Contexto del proyecto

El proyecto se enmarca en la necesidad de mejorar la movilidad dentro de aeropuertos modernos, caracterizados por largas distancias, flujos masivos de personas y altos niveles de actividad. En estas condiciones, los desplazamientos pueden resultar agotadores y confusos, especialmente para personas con movilidad reducida, familias con niños o viajeros que transportan equipaje voluminoso.

En este contexto, se propone el desarrollo de un robot móvil autónomo diseñado para asistir al pasajero durante su recorrido por la terminal aeroportuaria. El sistema tendrá la capacidad de guiar al usuario hacia su puerta de embarque mientras transporta su equipaje, proporcionando además información de orientación durante el trayecto. De esta forma, se busca reducir el esfuerzo físico del usuario y aumentar su comodidad y seguridad.

Stakeholders principales

Stakeholder	Interés	Rol
Pasajeros	Comodidad, seguridad, información clara	Usuarios finales del sistema
Aeropuerto (dirección operativa)	Eficiencia, satisfacción, reducción de costos	Propietarios y gestores
Seguridad aeroportuaria	Control, trazabilidad, integración con sistemas	Validación y supervisión
Equipo técnico del aeropuerto	Mantenimiento, integración de datos	Soporte operativo
Reguladores y autoridades	Cumplimiento normativo, seguridad pública	Validación legal
Fabricante del robot	Especificaciones, desempeño	Proveedor de tecnología

Restricciones operativas

- Entorno dinámico con miles de personas diarias
- Necesidad de operar 24/7 con alta disponibilidad
- Integración con infraestructura aeroportuaria existente
- Cumplimiento de regulaciones de seguridad aeroportuaria

Restricciones técnicas

- Presencia de interferencias electromagnéticas en la terminal
- Cobertura de conectividad irregular en ciertas zonas
- Necesidad de navegación autónoma en espacios complejos

Restricciones de negocio

- Presupuesto limitado para inversión inicial
- Tiempo de implementación acotado
- Rentabilidad esperada del proyecto

Dependencias externas

- Sistemas del aeropuerto: Información de vuelos, pasarelas, sistemas de seguridad
 - Infraestructura de red: WiFi, 5G, sistemas de comunicación
 - Proveedores de tecnología: Sensores, componentes robóticos, software de IA
 - Reguladores: Aprobaciones y certificaciones necesarias
-

3. Objetivos del proyecto

El objetivo principal de este proyecto es diseñar y desarrollar un prototipo funcional de robot móvil autónomo orientado a la asistencia de pasajeros en aeropuertos. Para ello, el sistema integrará diversas tecnologías propias del ámbito de la robótica y la inteligencia artificial, con el fin de proporcionar un servicio eficiente de guía y transporte de equipaje dentro de la terminal.

Desde el punto de vista tecnológico, el robot incorporará un sistema de navegación autónoma que le permitirá desplazarse de forma segura por el entorno, evitando obstáculos y adaptándose a la presencia de personas u otros elementos en movimiento. Asimismo, se implementará un sistema de visión artificial basado en OpenCV para el reconocimiento de personas. Paralelamente, se desarrollará una interfaz web que permitirá supervisar y controlar el robot de manera remota. Juntas, estas funcionalidades buscan demostrar el potencial de la robótica de servicio aplicada a entornos aeroportuarios mediante una solución innovadora, accesible y funcional.

Aspecto	Beneficio esperado
Experiencia del pasajero	Reducción de estrés, mejor orientación y mayor comodidad.
Operación del aeropuerto	Menos consultas al personal y mejor gestión de flujos.
Eficiencia económica	Menor dependencia de personal de información y mayor satisfacción de los pasajeros.
Accesibilidad	Mayor inclusión de personas con movilidad reducida o necesidades específicas.
Seguridad	Menor riesgo de accidentes y mejor monitorización de zonas de tránsito.
Datos operativos	Obtención de información sobre flujos y patrones de movimiento.
Imagen institucional	Diferenciación competitiva y percepción de innovación.

Métricas de éxito

- Adopción: al menos un 40% de los pasajeros utiliza el servicio durante el primer año.
- Satisfacción: puntuación igual o superior a 4.5 sobre 5 en encuestas de usuario.
- Disponibilidad: tiempo de funcionamiento igual o superior al 98%.

- Seguridad: cero accidentes graves relacionados con el robot.
 - Eficiencia: reducción de al menos un 20% en consultas al personal de información.
 - Rendimiento técnico: exactitud de navegación igual o superior al 95% y tiempo de respuesta igual o inferior a 2 segundos.
-

4. Definición de requisitos del proyecto

Requisitos funcionales

El sistema debe permitir que el robot guíe a los pasajeros por el aeropuerto, transporte equipaje, ofrezca información útil y se integre con los sistemas aeroportuarios. Estos requisitos recogen la función principal del robot y los servicios que aporta al usuario.

Requisito	Redacción
Guía autónoma de pasajeros	El robot debe recibir una solicitud de destino, calcular la ruta óptima y guiar al pasajero mediante instrucciones verbales y visuales, ajustando el recorrido en tiempo real según cambios operativos.
Transporte de equipaje	El robot debe contar con una plataforma de carga capaz de transportar hasta 30 kg de equipaje, asegurando la sujeción correcta y la estabilidad durante el movimiento.
Información de orientación	El robot debe proporcionar información sobre la terminal, como baños, cafeterías, tiendas y puertas, así como el estado de los vuelos, en al menos 8 idiomas.
Interacción con el usuario	El robot debe disponer de pantalla táctil y micrófono para recibir comandos, reconocer instrucciones de voz y permitir la confirmación de acciones por parte del usuario.
Integración con sistemas aeroportuarios	El robot debe conectarse con el sistema de información de vuelos del aeropuerto, sincronizar cambios de puerta y reportar incidencias al sistema de operaciones.

Requisitos técnicos

El sistema debe contar con una configuración hardware y software adecuada para navegar en interiores, localizarse con precisión e integrarse con otros sistemas del aeropuerto.

Categoría	Requisito
Hardware	Plataforma móvil con tracción diferencial o equivalente, sensores LiDAR, cámaras y sensores de proximidad, procesadores de última generación, conectividad WiFi y batería recargable con al menos 8 horas de autonomía.
Software	Sistema operativo Linux o ROS, algoritmos SLAM para mapeo y localización, IA para detección de obstáculos y predicción de movimientos, y API REST para integración con sistemas externos.

Requisitos de usuario

El robot debe adaptarse a distintos perfiles de usuario, garantizando facilidad de uso, accesibilidad y apoyo al personal del aeropuerto.

Usuario	Requisito
Pasajeros regulares	Interfaz simple sin necesidad de entrenamiento previo, comunicación clara en su idioma nativo y posibilidad de cancelar o cambiar el destino en cualquier momento.
Personas con discapacidad	Soporte de audio para usuarios con discapacidad visual, pantalla de alto contraste y posibilidad de control manual si es necesario.
Personal del aeropuerto	Interfaz de administración para monitoreo de flota, capacidad de control manual en emergencias y acceso a reportes de operación y mantenimiento.

Restricciones técnicas

El sistema debe funcionar correctamente en las condiciones propias de un aeropuerto, donde existen limitaciones de navegación, conectividad y cambios constantes en la operación.

Restricción	Descripción
Navegación en interiores	No puede depender de GPS y debe usar mapas internos.
Interferencias electromagnéticas	Debe funcionar en zonas con interferencias, como áreas de seguridad o escáneres.
Conectividad limitada	Debe seguir operando aunque no haya WiFi en todos los puntos.
Cambios dinámicos	Debe adaptarse a cierres de zonas, cambios de puerta y eventos operativos.

Priorización de requisitos

No todos los requisitos tienen la misma importancia. La guía autónoma, la seguridad y el transporte de equipaje son las funciones más críticas, mientras que otros elementos aportan valor añadido pero no son esenciales para el funcionamiento básico.

Requisito	Prioridad	Justificación
Guía autónoma a puertas de embarque	Crítica	Es la función principal del sistema.
Transporte de equipaje	Crítica	Es uno de los valores diferenciales del robot.
Detección de obstáculos	Crítica	Es esencial para la seguridad de pasajeros y del propio robot.
Información en múltiples idiomas	Alta	Es necesaria para atender a usuarios internacionales.

Integración con sistema de vuelos	Alta	Permite mantener la información actualizada.
Información de cafeterías y tiendas	Media	Aporta valor añadido, pero no es esencial.

Criterios de aceptación

Cada requisito debe validarse mediante pruebas funcionales, de robustez, de seguridad y de documentación. El sistema se considerará correcto si ejecuta sus funciones de forma fiable, responde adecuadamente ante errores, no introduce riesgos adicionales y mantiene un comportamiento coherente con lo esperado.

5. Diseño

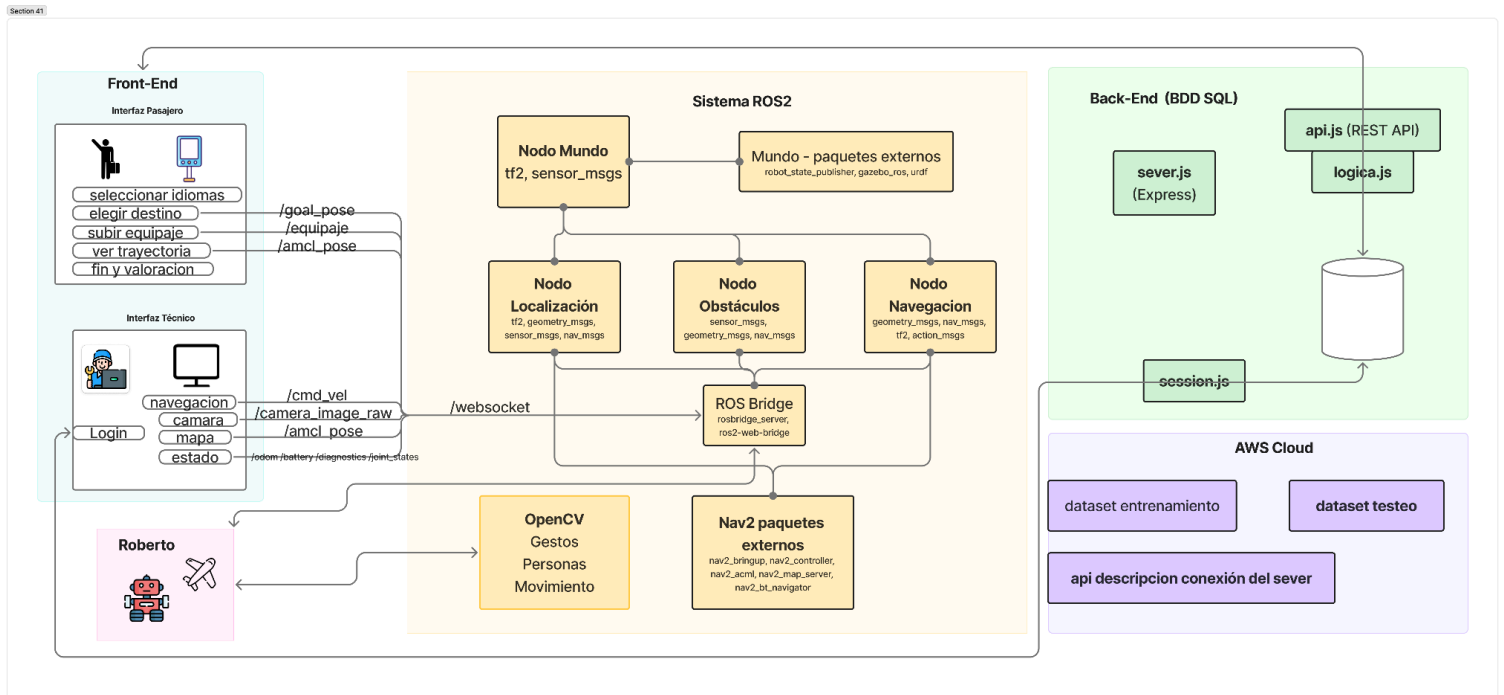


Figura : Arquitectura completa

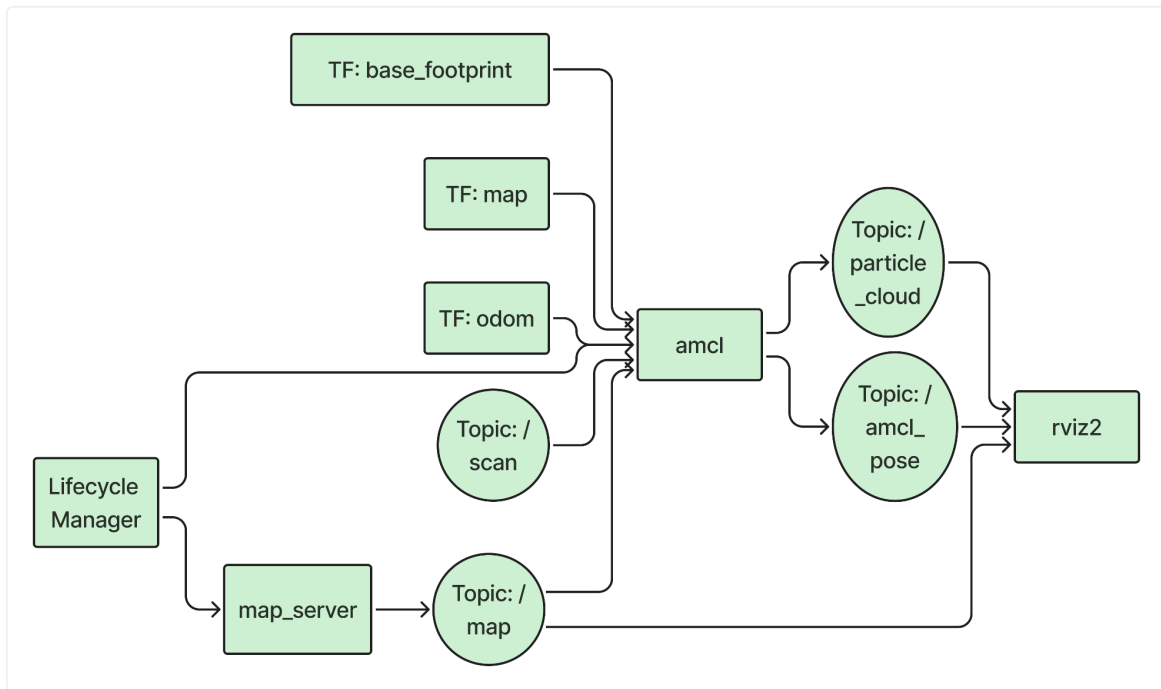


Figura : Diseño de paquete de localización

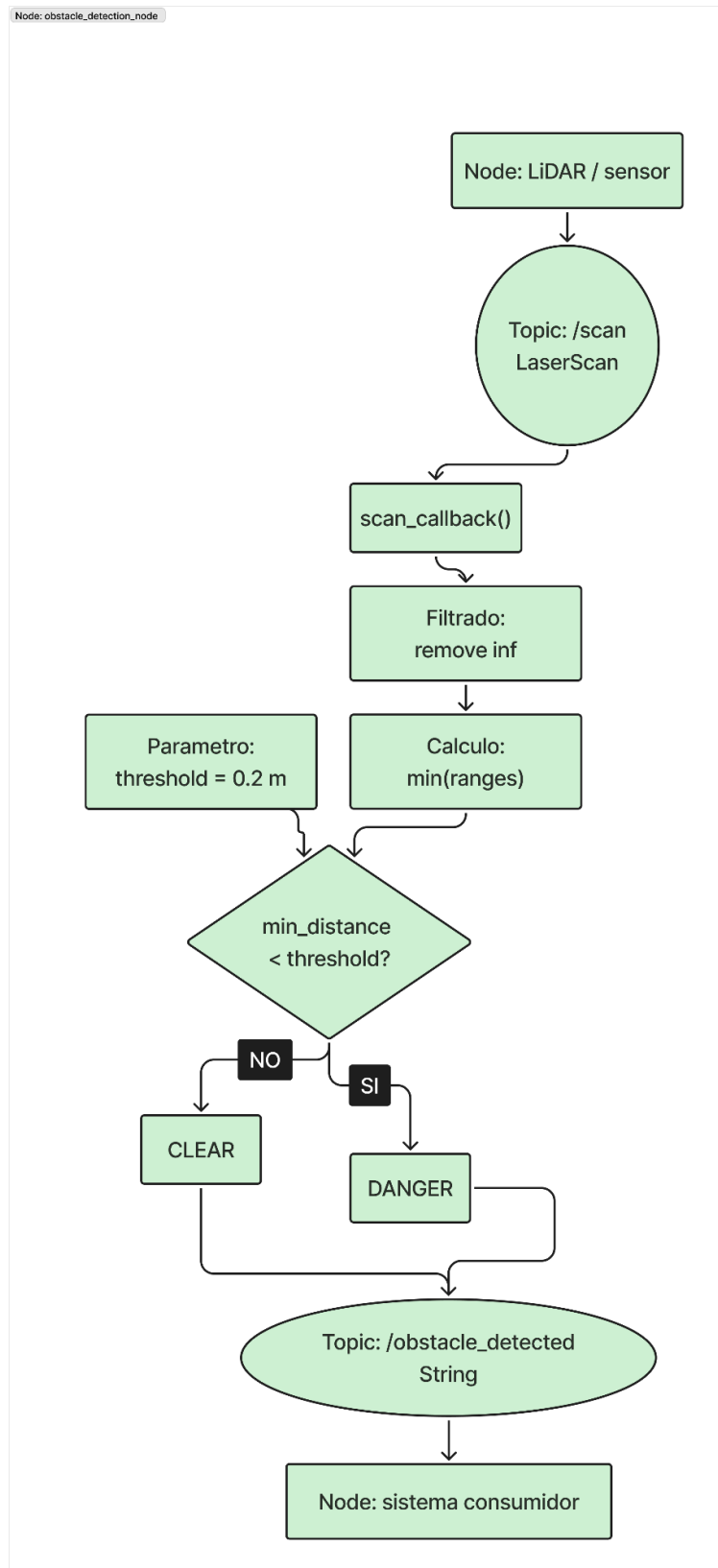


Figura : Diseño de paquete LiDAR

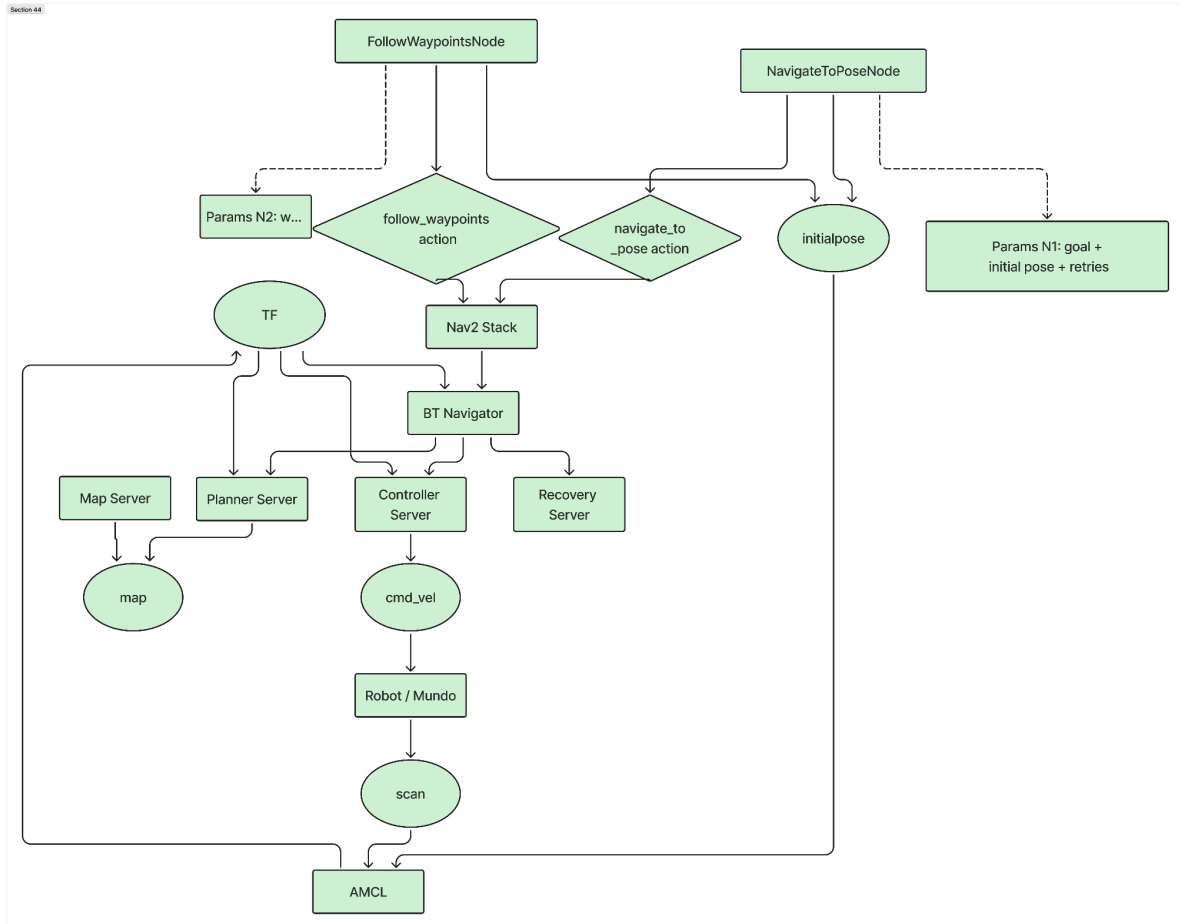
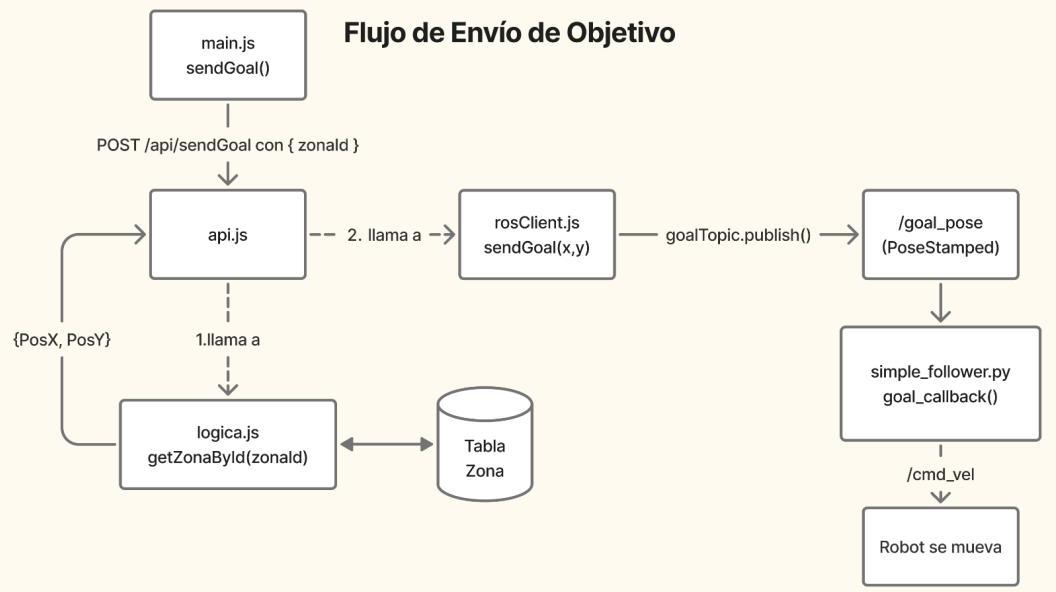


Figura : Diseño de paquete navegación

Visualización Web del Mapa y Navegación del Robot en Tiempo Real

Flujo de Envío de Objetivo



Flujo de Telemetría en Tiempo Real

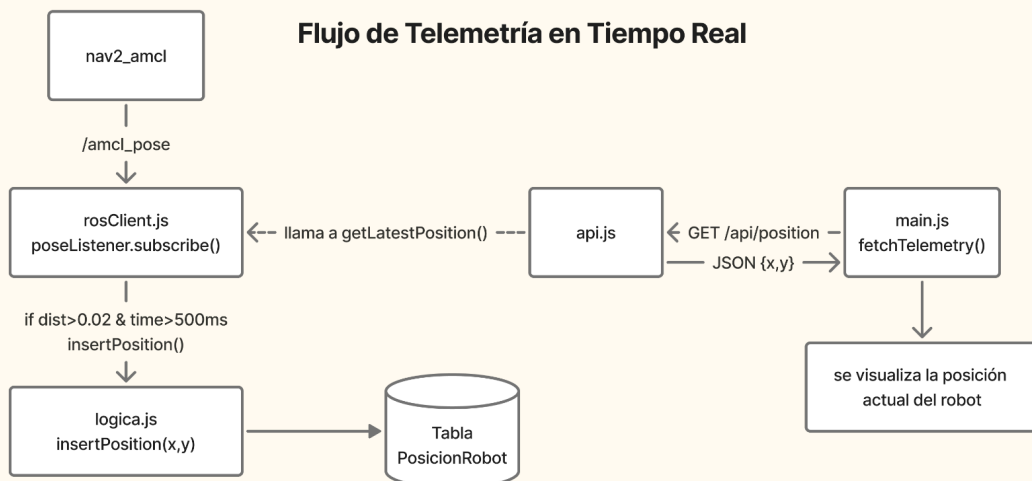


Figura : Diseño de flujo de visualizacion del mapa

Gestión de conexión del robot

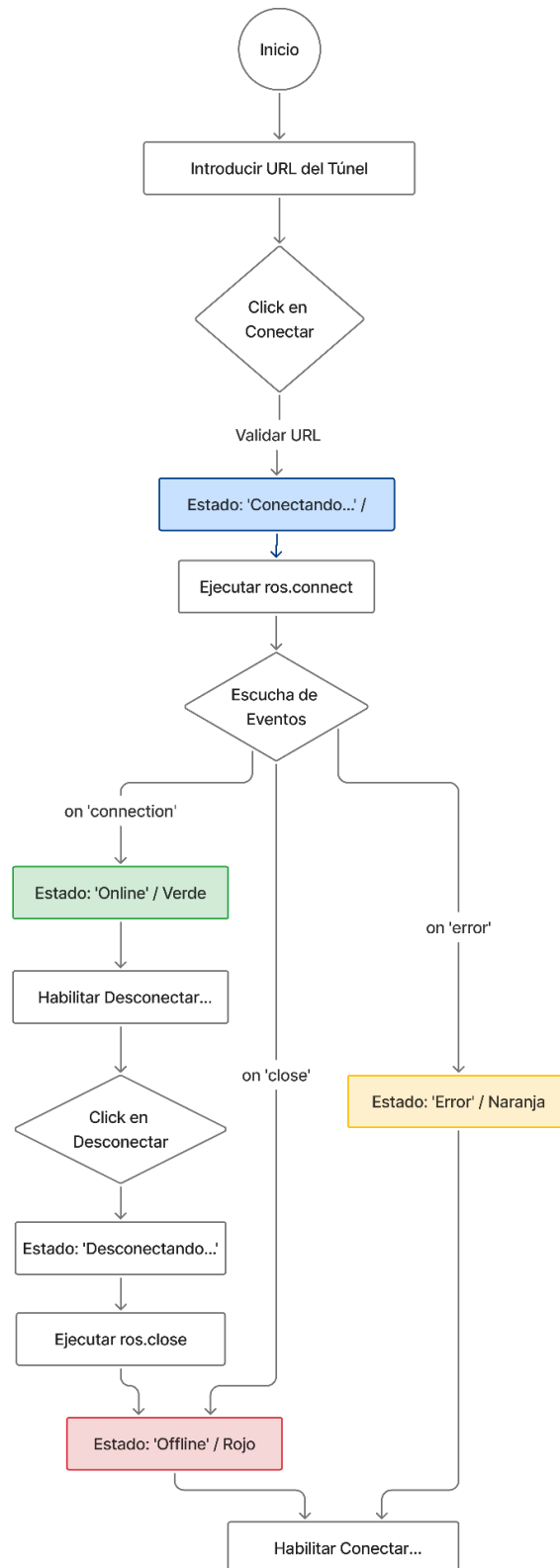


Figura : Diseño de conexión del Robot

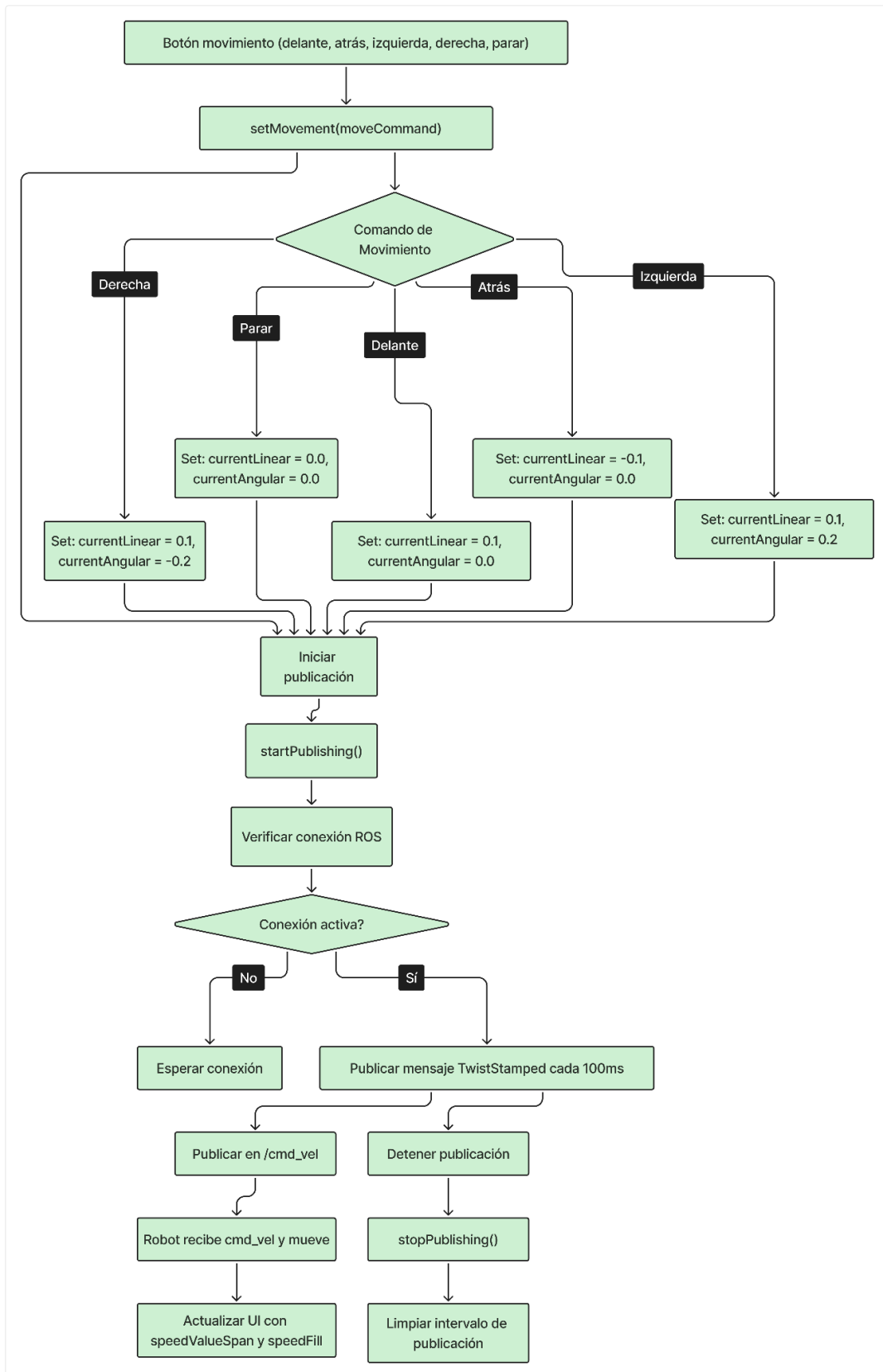


Figura : Diseño de Control Manual del Robot

- Diseño del front-end:

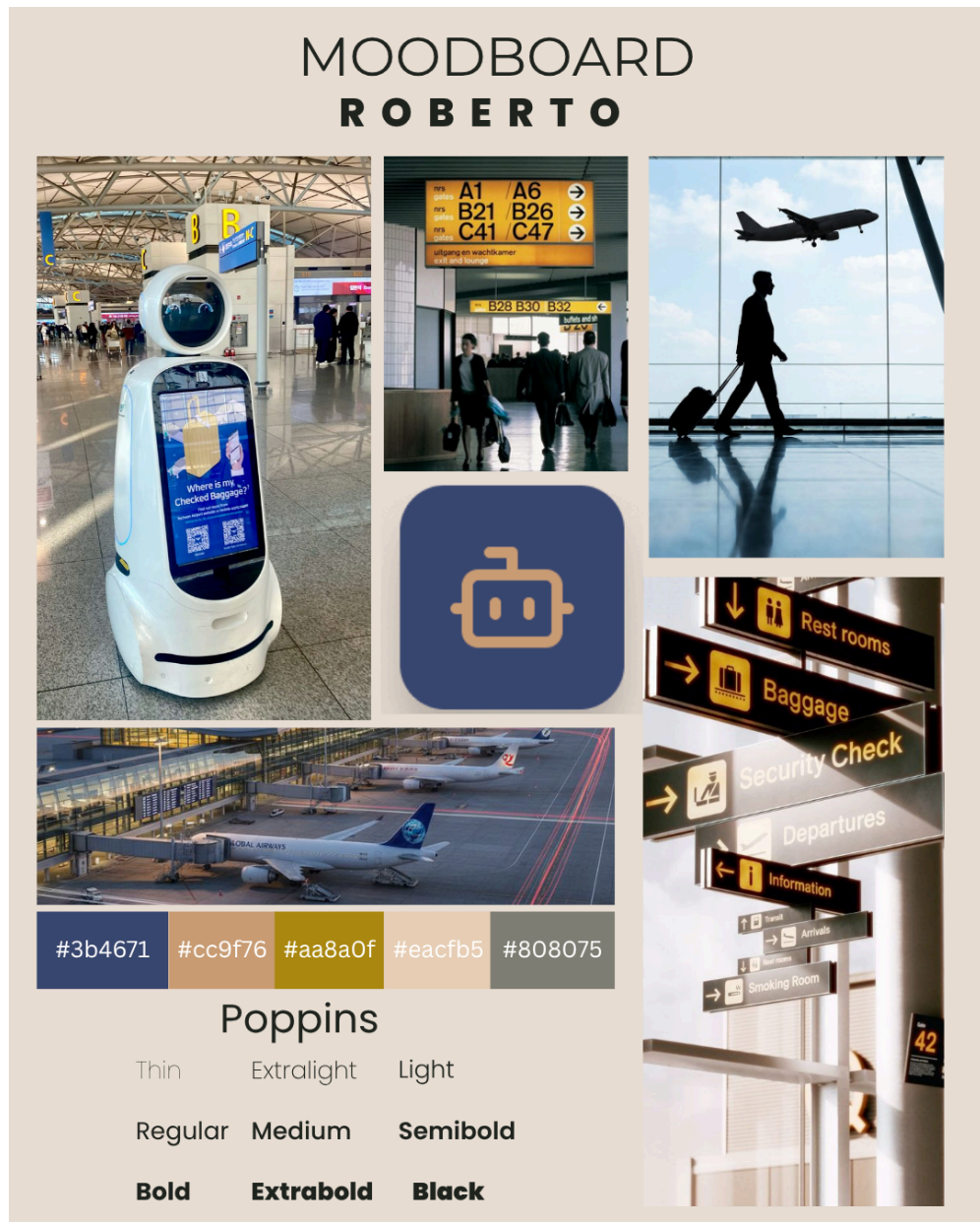


Figura : MoodBoard



Figura : Pantalla inicial

Pantalla de bienvenida del sistema ROBERTO, donde el usuario puede seleccionar el idioma de la interfaz antes de comenzar. Presenta una interfaz clara e intuitiva con opciones multilingües y un acceso directo al sistema mediante el botón "Comenzar", facilitando una experiencia de inicio rápida y accesible.

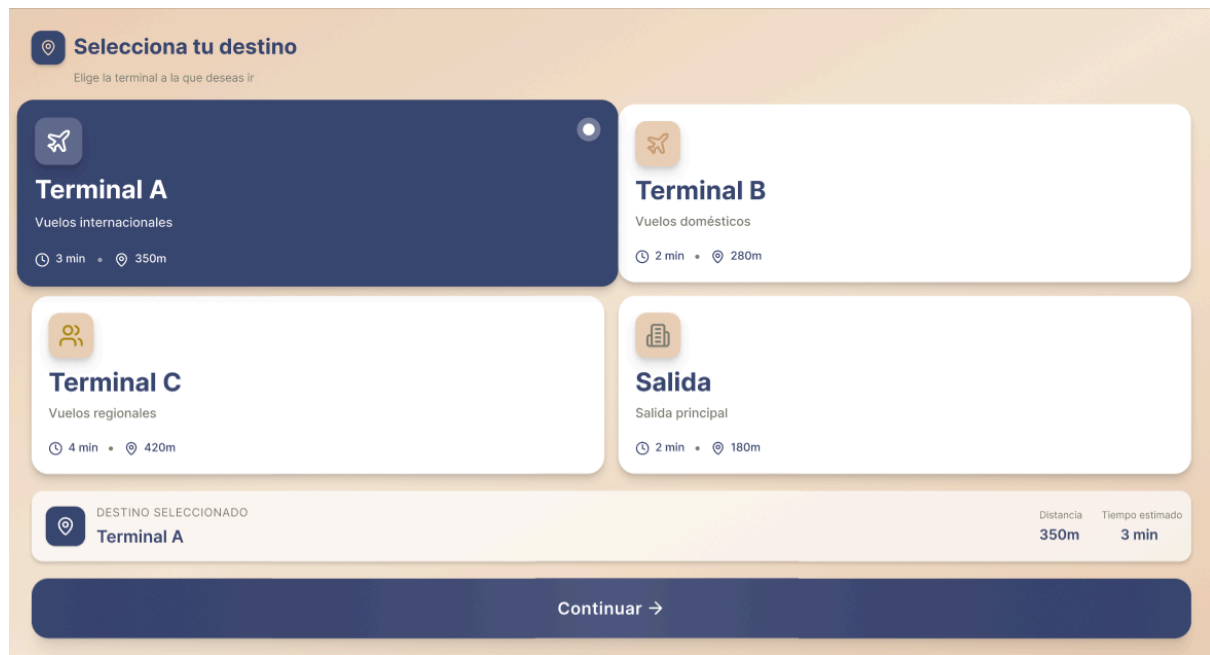


Figura : Selecccion de terminal

Pantalla donde el usuario elige su destino dentro del aeropuerto, mostrando distintas terminales y salidas con información relevante como tiempo estimado y distancia. La interfaz permite una selección rápida e intuitiva, destacando el destino elegido y ofreciendo un botón de continuación para iniciar la navegación.

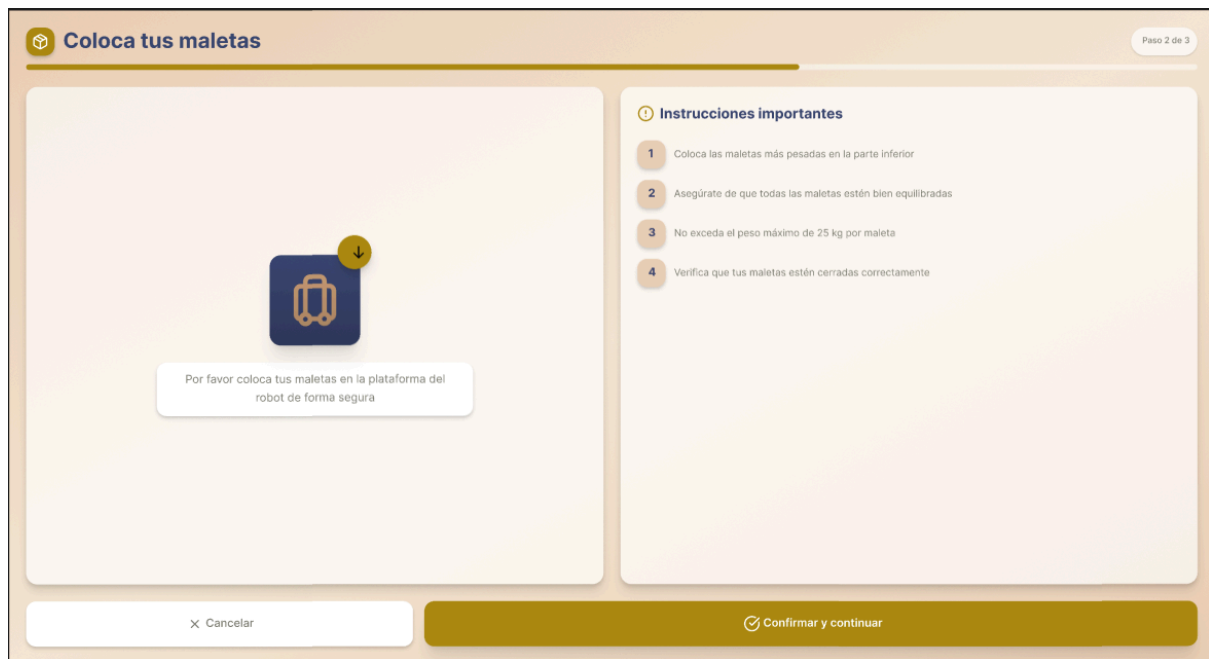


Figura: Cargar maletas

Pantalla en la que el usuario debe colocar sus maletas en el robot antes de iniciar el transporte. Incluye instrucciones claras para asegurar una correcta distribución y seguridad del equipaje, así como opciones para cancelar o confirmar la carga y continuar con el proceso.



Figura : En camino

Pantalla que muestra el estado del robot durante el trayecto hacia el destino seleccionado. Incluye información en tiempo real como la posición en la ruta, velocidad, distancia restante, tiempo estimado de llegada y porcentaje de progreso, permitiendo al usuario seguir el recorrido de forma clara y segura.



Figura: Llegado

Pantalla final que confirma la llegada del robot al destino, mostrando un resumen del viaje (duración, distancia y estado) y los próximos pasos para el usuario. Además, permite valorar

la experiencia del servicio, cerrando el proceso de forma clara, interactiva y orientada a la mejora continua.

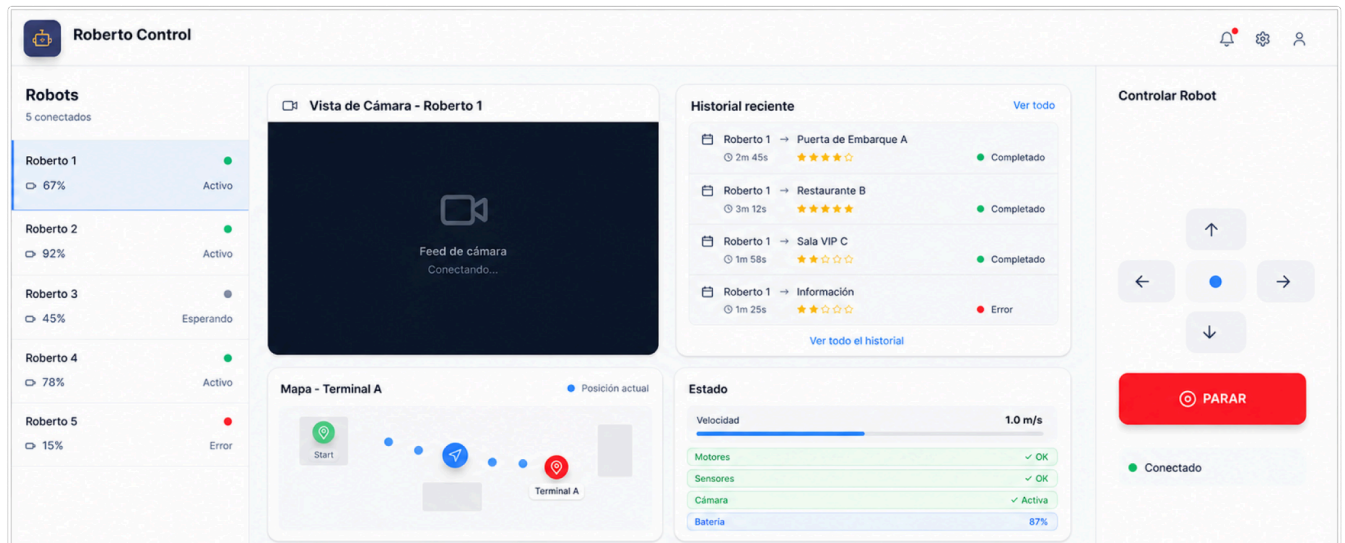


Figura: Interfaz del técnico

Interfaz web destinada al control y supervisión de los robots, donde el técnico puede visualizar en tiempo real la cámara, la posición en el mapa, historial de interacciones, y el estado del sistema (velocidad, sensores, batería). Además, permite el control manual del robot mediante comandos direccionales y la gestión de múltiples robots desde un único panel.

- **Diseño del back-end:**

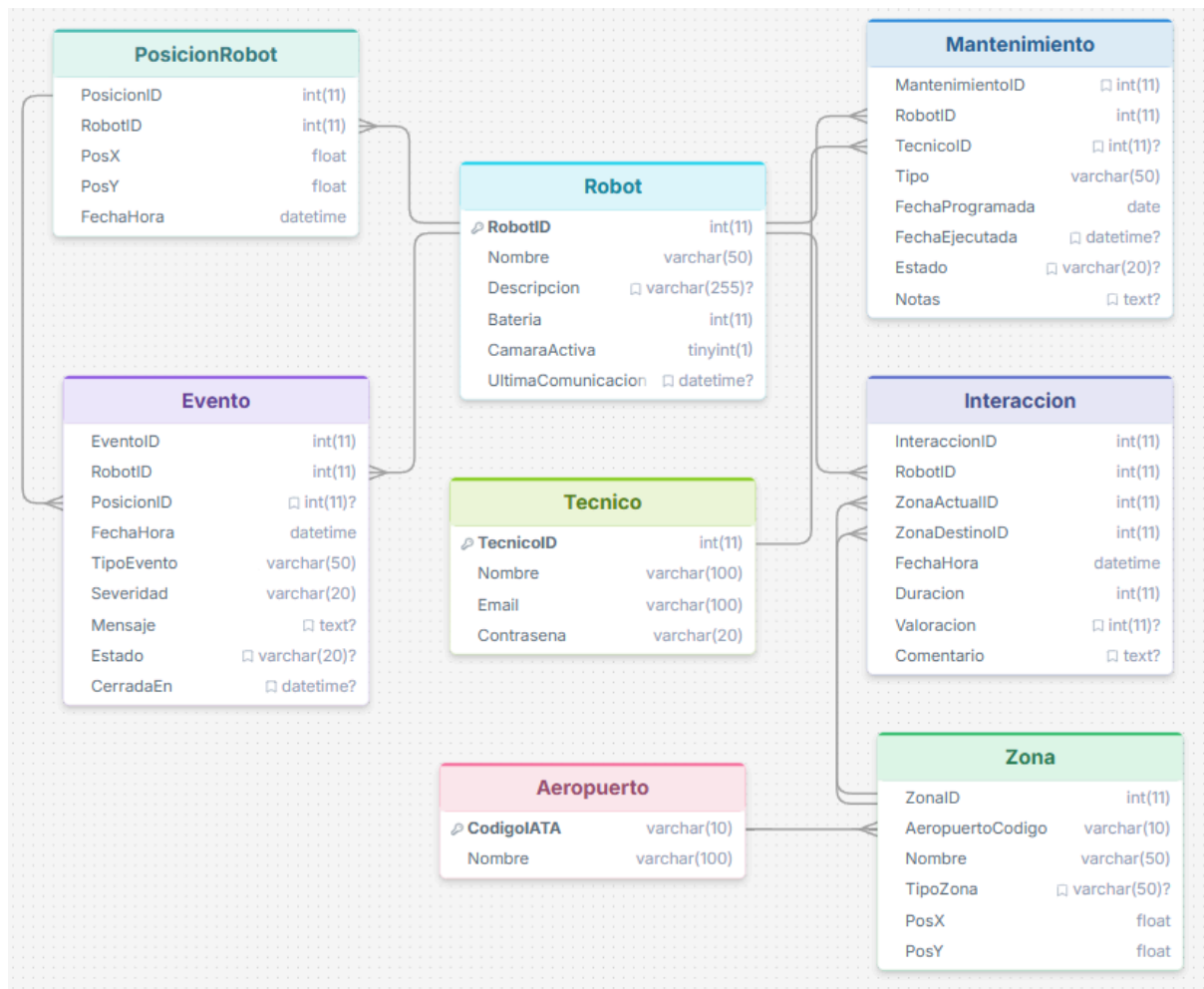


Figura: BBDD

El esquema detalla la arquitectura de datos diseñada para soportar tres ejes operativos principales: la telemetría y control de incidencias en tiempo real (**PosicionRobot**, **Evento**), la gestión del estado y reparaciones del equipo (**Mantenimiento**, **Tecnico**), y la inteligencia de negocio orientada al pasajero (**Interaccion**). Destaca el modelado optimizado de las interacciones humano-robot, las cuales se vinculan a áreas lógicas del aeropuerto (**ZonaActualID** y **ZonaDestinoID**) de forma independiente al rastreo milimétrico de la posición. Esta arquitectura permite al aeropuerto analizar flujos de usuarios, generar mapas de calor y detectar puntos de desorientación para mejorar la señalización física, asegurando a su vez un alto rendimiento en las consultas de la base de datos.

- **Tecnologías necesarias:**

El sistema desarrollado se basa en **ROS 2 Jazzy** como middleware robótico, encargado de la comunicación entre nodos y la gestión de los distintos módulos del sistema. Para la simulación del entorno del aeropuerto se ha utilizado **Gazebo**, donde se ha integrado un robot equipado con sensores, especialmente un **LiDAR**, utilizado para la detección del entorno y la navegación.

La localización y navegación del robot se han implementado utilizando los paquetes de ROS 2 correspondientes, apoyándose en los datos del sensor LiDAR y en mapas previamente generados. Para la gestión de mapas se ha utilizado **map_server**, permitiendo cargar y utilizar el entorno para la navegación del robot.

Durante el desarrollo, la visualización del estado del robot, su posición y los datos de sensores se ha realizado mediante **RViz2**, herramienta fundamental para la validación del sistema. El desarrollo de nodos y lógica se ha llevado a cabo en **Python**, siguiendo la estructura de paquetes de ROS 2 y utilizando **colcon** como herramienta de compilación.

En cuanto a la interfaz de usuario, se ha desarrollado una aplicación web para la interacción con el sistema, permitiendo la selección de destinos y la monitorización del robot, conectándose con el sistema robótico mediante los nodos de ROS 2.

6. Implementación

- Estructura de carpetas del proyecto:

```
roberto/                                <-- Carpeta raíz del repositorio (clonado en src/)
├── README.md
├── roberto/                             <-- Metapaquete
│   ├── CMakeLists.txt
│   └── package.xml
├── roberto_lidar/                       <-- Paquete de funcionalidad (Python)
│   ├── launch/
│   ├── resource/
│   ├── rviz/
│   ├── test/
│   ├── package.xml
│   ├── setup.cfg
│   └── setup.py
├── roberto_mundo/                       <-- Paquete de funcionalidad (C++/CMake)
│   ├── include/
│   ├── launch/
│   ├── maps/
│   ├── models/
│   ├── params/
│   ├── rviz/
│   ├── src/
│   ├── urdf/
│   ├── worlds/
│   ├── CMakeLists.txt
│   └── package.xml
├── roberto_nav_punto/                   <-- Paquete de funcionalidad (Python)
│   ├── config/
│   ├── launch/
│   ├── resource/
│   ├── roberto_nav_punto/
│   ├── rviz/
│   ├── test/
│   ├── package.xml
│   ├── setup.cfg
│   └── setup.py
└── roberto_nav_ruta/                   <-- Paquete de funcionalidad (Python)
    ├── launch/
    ├── param/
    ├── resource/
    ├── roberto_nav_ruta/
    ├── rviz/
    ├── test/
    ├── package.xml
    ├── setup.cfg
    └── setup.py
```

Figura: Estructura de carpetas

6.1 Listado de componentes implementados

Se han implementado los siguientes componentes que conforman el sistema integral de navegación autónoma del robot Roberto:

Componente	Lenguaje	Función principal
Nodo de detección de obstáculos roberto_lidar	Python	Procesar datos del sensor LiDAR y generar alertas de obstáculos
Simulación y modelos roberto_mundo	C++/CMake	Proporcionar entorno de simulación en Gazebo con modelos 3D y mapas
Localización adaptativa roberto_nav_punto	Python	Estimar la pose del robot mediante AMCL combinando odometría y observaciones LiDAR
Navegación por waypoints y objetivos roberto_nav_ruta	Python	Ejecutar secuencias de waypoints o navegar hacia un objetivo único
Metapaquete coordinador roberto	—	Orquestrar la carga conjunta de todos los componentes del sistema
Interfaz web de visualización frontend/main.js	Javascript	Renderizar el mapa (ROS2D), mostrar la posición del robot y aplicar interpolación
Servidor web backend/server.js	Javascript	Configurar el servidor Express y servir la interfaz web
API REST backend/api.js	Javascript	Gestionar las peticiones HTTP entre frontend, base de datos y ROS
Cliente ROS (middleware) backend/rosClient.js	Javascript	Conectarse a ROS Bridge, suscribirse a /amcl_pose y publicar metas en /goal_pose
Persistencia de datos backend/logica.js	JavaScript	Consultas y almacenamiento en MySQL
Nodo deteccion gestos roberto_gesture_detection	Python	Identifica gestos para acercarse a un pasajero

Componente	Lenguaje	Función principal
Nodo de detección de personas roberto_person_detection	Python	Identificar pasajeros mediante visión artificial y procesar su presencia en tiempo real (OpenCV)

6.2 Detalle de la implementación de componentes

El sistema de navegación autónoma se compone de cuatro paquetes funcionales principales más un metapaquete coordinador. El desarrollo modular del proyecto Roberto facilita la integración de nuevas funcionalidades sin afectar los componentes existentes.

6.2.1 Nodo de detección de obstáculos (roberto_lidar)

El paquete roberto_lidar implementa toda la lógica de procesamiento de datos del sensor LiDAR. Este componente funciona de forma continua, capturando mediciones de distancia del entorno y transformándolas en alertas de peligro para el sistema de navegación.

El paquete contiene nodos Python que suscriben al tópico de escaneo LiDAR (/scan) y procesan las mediciones brutas. Aplica filtros para eliminar lecturas anómalas causadas por reflexiones incorrectas o interferencias electromagnéticas. Transforma las mediciones de coordenadas polares (rango y ángulo) a coordenadas cartesianas (x, y) para obtener posiciones espaciales precisas de los obstáculos detectados.

El nodo mantiene un registro de la distancia mínima a obstáculos en diferentes direcciones alrededor del robot. Cuando alguna distancia cae por debajo de un umbral de seguridad configurado, se genera una alerta que otros componentes pueden utilizar para activar comportamientos de emergencia, como ralentizar o detener el robot.

6.2.2 Simulación y modelado (roberto_mundo)

El paquete roberto_mundo proporciona el entorno de simulación completo basado en Gazebo. Contiene toda la definición del robot, los escenarios de prueba y la configuración física necesaria para validar el comportamiento del sistema sin hardware real.

El directorio urdf/ contiene los archivos de descripción del robot en formato URDF (Unified Robot Description Format), definiendo la estructura mecánica, articulaciones y sensores del robot Roberto. Los modelos 3D se encuentran en models/, mientras que los mundos de simulación se definen en worlds/. Cada mundo especifica obstáculos, superficies, iluminación y propiedades físicas de forma mínima para mejorar el rendimiento.

La carpeta maps/ almacena los mapas estáticos en formato PGM (imagen) acompañados de metadatos YAML que definen resolución, origen y escala. Los parámetros de configuración de Gazebo y otros componentes residen en params/, permitiendo ajustar dinámicamente el comportamiento del simulador.

6.2.3 Localización adaptativa (roberto_nav_punto)

El paquete `roberto_nav_punto` implementa el algoritmo de localización adaptativa por Monte Carlo (AMCL). Este componente estima continuamente dónde se encuentra el robot dentro del mapa global, combinando información de odometría y observaciones del sensor LiDAR.

AMCL estima dónde está el robot usando un conjunto de "hipótesis" (partículas). Cada hipótesis propone una posición y orientación posibles. El algoritmo elige cuál es la más probable comparando lo que el robot debería ver desde cada posición con lo que realmente ve según sus sensores.

6.2.4 Navegación por waypoints y objetivos (roberto_nav_ruta)

El paquete `roberto_nav_ruta` implementa la capacidad de navegar hacia objetivos, ya sean puntos únicos o secuencias de waypoints predefinidas. Calcula trayectorias y las ejecuta mediante comandos de velocidad, adaptándose dinámicamente a cambios en el entorno.

El nodo realiza planificación global utilizando algoritmos de grafos sobre el mapa estático proporcionado por `roberto_mundo`. Consulta la información de obstáculos cercanos proveniente de `roberto_lidar` para evitar colisiones dinámicas. El resultado es una secuencia de waypoints que respeta tanto la geometría del entorno como los límites físicos del robot.

Para navegación hacia un objetivo único, el sistema planifica una trayectoria directa y la ejecuta. Para rutas compuestas por múltiples waypoints almacenados en `param/`, el nodo navega secuencialmente hacia cada punto, evaluando cuando alcanza cada objetivo para proceder al siguiente. Después de completar todos los waypoints, la ruta termina.

6.2.5 Metapaquete Roberto

El metapaquete simplifica la gestión de dependencias complejas y permite un lanzamiento coordinado del sistema.

El archivo `package.xml` declara explícitamente las dependencias de todos los paquetes: `roberto_lidar`, `roberto_mundo`, `roberto_nav_punto` y `roberto_nav_ruta`. Esto permite a herramientas como `rosdep` resolver automáticamente qué necesita instalarse.

El metapaquete coordina la inicialización de todo el sistema en el orden correcto: primero la simulación (`roberto_mundo`), luego la localización (`roberto_nav_punto`), seguida de la detección de obstáculos (`roberto_lidar`), y finalmente los nodos de navegación (`roberto_nav_ruta`) según sea necesario.

6.3 Pagina web (Tecnico)

6.3.1 Interfaz web de visualización (main.js)

El archivo `main.js` implementa la lógica de la interfaz web encargada de representar el mapa y el estado del robot en tiempo real. Utiliza ROS2D para renderizar el mapa y mostrar la posición del robot mediante un marcador visual.

Este componente consulta periódicamente la posición del robot a través del endpoint `/api/position` y actualiza la visualización aplicando interpolación, lo que permite un movimiento suave en el mapa. Además, gestiona la interacción del usuario, permitiendo seleccionar destinos y enviar objetivos de navegación al backend mediante peticiones HTTP.

6.3.2 Servidor web (server.js)

El archivo `server.js` configura el servidor Express que actúa como punto de entrada del sistema web. Se encarga de servir los archivos estáticos del frontend y de habilitar la comunicación mediante una API REST.

Además, inicializa la conexión con ROS a través de `rosClient.js`, garantizando que la telemetría del robot esté disponible antes de que los clientes web comiencen a realizar peticiones.

6.3.3 API REST (api.js)

El módulo `api.js` define los endpoints que permiten la comunicación entre el frontend, el sistema ROS y la base de datos. Actúa como capa intermedia que traduce las peticiones HTTP en acciones del sistema.

Entre sus funciones principales se encuentran la obtención de la posición actual del robot (`/api/position`), el envío de objetivos de navegación (`/api/sendGoal`) y la consulta de información almacenada en la base de datos, como zonas o estado del robot.

6.3.4 Cliente ROS (rosClient.js)

El módulo `rosClient.js` gestiona la comunicación con ROS 2 mediante WebSockets utilizando ROS Bridge. Se encarga de suscribirse al tópico `/amcl_pose` para recibir la posición del robot en tiempo real y almacenarla en memoria para su acceso rápido desde la API.

Asimismo, permite publicar objetivos de navegación en el tópico `/goal_pose`, convirtiendo las peticiones del sistema web en comandos comprensibles por el robot.

6.3.5 Persistencia de datos (logica.js)

El módulo `logica.js` gestiona la interacción con la base de datos MySQL. Proporciona funciones para consultar información relevante, como las coordenadas de las zonas de navegación, y para almacenar datos generados por el sistema.

Entre sus responsabilidades se incluye el registro del historial de posiciones del robot en la tabla `PosicionRobot`, lo que permite realizar análisis posteriores y mantener trazabilidad del movimiento.

6.3.6 Nodo de control del robot (simple_follower.py)

El nodo simple_follower.py implementa un controlador básico de navegación en ROS 2. Recibe objetivos de navegación desde el tópico /goal_pose y calcula los comandos de velocidad necesarios para alcanzar el destino.

Mediante un control proporcional, ajusta la velocidad lineal y angular del robot, publicando en /cmd_vel hasta que el objetivo es alcanzado.

6.4 Página web (Usuario)

6.3.1 Pantalla inicial y selección de idioma

La funcionalidad de pantalla inicial se implementa principalmente en el archivo index.html junto con la lógica definida en main.js, encargada de gestionar la selección de idioma y la navegación inicial del usuario.

Este componente carga dinámicamente los textos desde archivos JSON de idiomas, permitiendo adaptar la interfaz según la selección realizada por el usuario. La preferencia de idioma se almacena durante la sesión para mantener la consistencia en toda la aplicación.

Además, se incluye un mecanismo de acceso discreto al área técnica mediante un botón oculto integrado en la interfaz. Este botón redirige al usuario a la pantalla de login del operador sin afectar la experiencia del pasajero.

6.3.2 Selección de destino

La funcionalidad de selección de destino se implementa en la vista destination.html y su lógica asociada en main.js. Permite al usuario elegir un destino dentro del entorno disponible.

Una vez seleccionado, el sistema envía la información al backend mediante un endpoint específico. Este backend procesa la solicitud y la transmite a un nodo ROS2 encargado de gestionar la navegación del robot.

El sistema garantiza la correcta comunicación entre la interfaz web, el backend y ROS2, iniciando el desplazamiento del robot hacia el destino seleccionado.

6.3.3 Navegación en curso

La pantalla de navegación, implementada en html y gestionada mediante main.js, muestra al usuario el estado del trayecto en tiempo real.

Este módulo consulta periódicamente el estado del robot a través de un endpoint de navegación, obteniendo información como el progreso del recorrido, la velocidad actual y el tiempo estimado restante. Estos datos se actualizan dinámicamente en la interfaz, proporcionando una visualización continua y coherente del desplazamiento.

La lógica se apoya en un nodo ROS2 que publica el estado del robot, permitiendo sincronizar la información entre el sistema físico y la aplicación web.

6.3.4 Pantalla de llegada y valoración

La funcionalidad de llegada se implementa en html, con soporte de main.js para gestionar la interacción del usuario.

Cuando el sistema detecta que el robot ha alcanzado el destino (a través del backend y ROS2), se muestra automáticamente la pantalla final. En esta vista, el usuario puede introducir una valoración del servicio y añadir comentarios.

Los datos introducidos se envían al backend mediante un endpoint de valoración, donde son procesados e insertados en la base de datos para su almacenamiento y posterior análisis.

7. Verificación de las funcionalidades implementadas

7.1 Listado de pruebas realizadas

Se han ejecutado las siguientes pruebas de funcionalidad para validar la implementación del sistema autónomo de navegación:

- **Prueba 1: Navegación autónoma**
 - **Objetivo:** Validar la capacidad del robot de desplazarse hacia objetivos predefinidos.
 - **Componente evaluado:** Planificador de rutas y control de movimiento.
- **Prueba 2: Detección de obstáculos con LiDAR**
 - **Objetivo:** Verificar la detección en tiempo real de obstáculos estáticos y dinámicos.
 - **Componente evaluado:** Lectura de sensor LiDAR.
- **Prueba 3: Localización (AMCL)**
 - **Objetivo:** Confirmar la sincronización de pose y corrección de odometría.
 - **Componente evaluado:** Map server y algoritmo AMCL.
- **Conexión websocket**
 - **Objetivo:** Garantizar una conexión local estable y segura entre la web y el robot mediante una gestión de estados con feedback visual en tiempo real.
 - **Componente evaluado:** Gestión de interfaz: control lógico de botones (Conectar/Desconectar) y prevención de estados inconsistentes.
- **Estado topics**
 - **Objetivo:** Asegurar que los topics como odometría, control de movimiento y sensores estén activos y funcionando.
 - **Componente evaluado:** La interfaz muestra la última medida obtenida.
- **Control manual**

- **Objetivo:** Validar que el robot pueda ser controlado manualmente.
- **Componente evaluado:** Interfaz manual, joystick o consola; uso correcto de topics.
- **Login técnico**
 - **Objetivo:** Validar que se prohíbe el acceso no autorizado.
 - **Componente evaluado:** Sistema de autenticación usando la base de datos.
- **Visualización web del mapa y navegación**
 - **Objetivo:** Validar que el sistema permite mostrar el mapa, actualizar la posición del robot en tiempo real y enviar objetivos de navegación desde la interfaz web.
 - **Componente evaluado:** Interfaz web, API REST, comunicación con ROS2.
- **Historial en la web**
 - **Objetivo:** Monitorizar el registro de misiones y la satisfacción del usuario mediante filtros dinámicos y exportación de datos.
 - **Componente evaluado:** Base de datos MySQL (JOINS), API REST, lógica de paginación y exportación CSV.
- **Prueba: Detección de personas (OpenCV)**
 - **Objetivo:** Validar la identificación visual de pasajeros en tiempo real y la publicación de su estado.
 - **Componente evaluado:** Nodo de visión artificial, cámara y procesamiento con Haar Cascades.
- **Prueba: Detección de gestos (MediaPipe)**
 - **Objetivo:** Validar la detección del gesto de llamado (pulgar extendido con puño cerrado) y la publicación de comandos de velocidad al robot.
 - **Componente evaluado:** Nodo de visión artificial, cámara y procesamiento con MediaPipe HandLandmarker.

7.2 Detalle de las pruebas ejecutadas

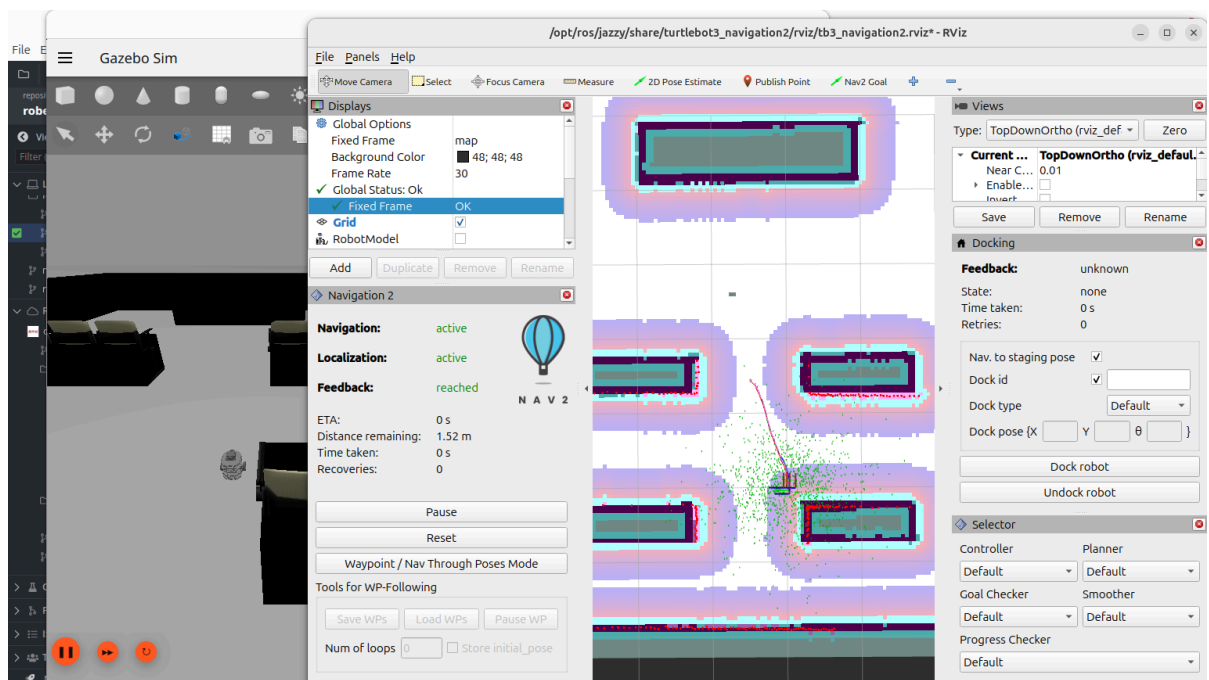
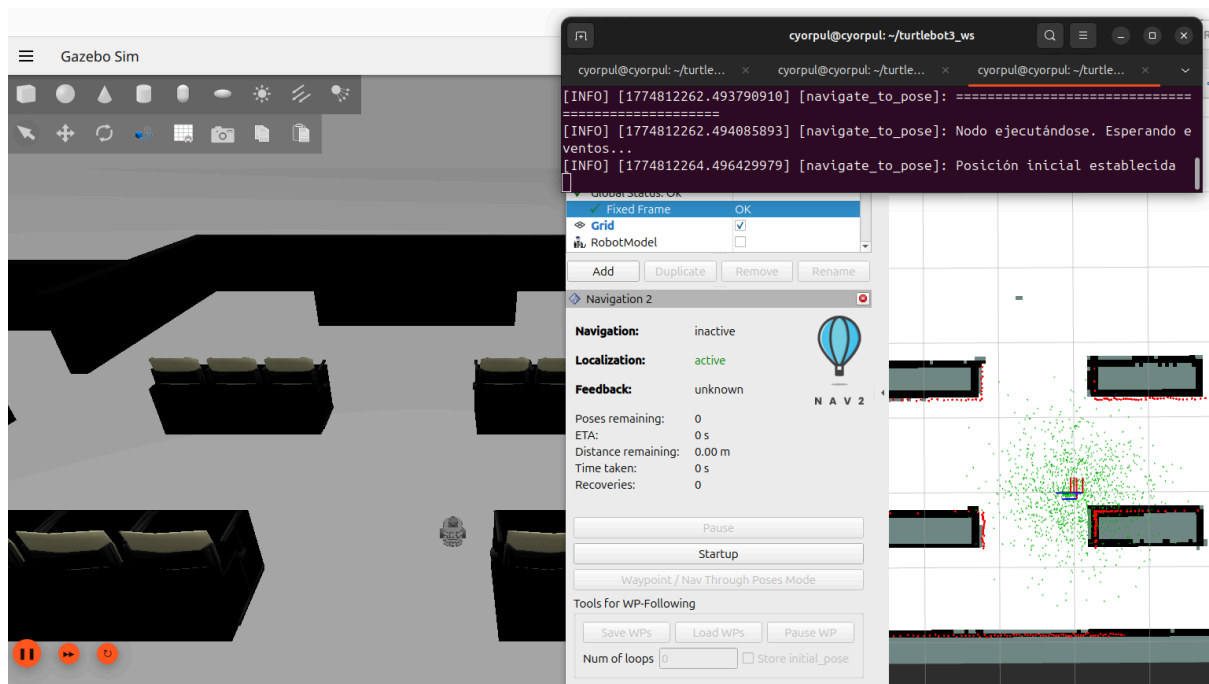
7.2.1 Navegación autónoma

- **Objetivo:** Validar que el robot establece correctamente su pose inicial, genera un mapa de costos y planifica una trayectoria óptima hacia el objetivo.
- **Procedimiento:**
 - El robot se inicializa en el entorno simulado de Gazebo.
 - Se genera un costmap que representa la dificultad de atravesar distintas zonas.
 - El planificador de ruta calcula una trayectoria óptima hacia el objetivo predefinido.
 - Se valida que el robot se desplaza de manera autónoma evitando colisiones.
- **Evidencias:**

Las Figuras 13 y 14 muestran el proceso de navegación en el entorno simulado del aeropuerto. La pose inicial se establece dentro del mapa con coordenadas (x, y, θ) definidas, y el cost map genera zonas de mayor coste alrededor de los obstáculos detectados, representadas mediante gradientes de color que indican la dificultad de atravesar cada región. El planificador genera una

trayectoria que considera tanto la distancia al objetivo como la evitación de zonas de alto coste, optimizando el camino a seguir.

El robot sigue la trayectoria planificada mediante actuadores de movimiento, alterando su velocidad angular y lineal según los waypoints calculados por el planificador global. La funcionalidad del sistema permite que el robot se desplace de forma iterativa: calcula la trayectoria global, la descompone en objetivos locales, y ajusta el movimiento mediante control reactivo en función de cambios en el entorno. Este comportamiento cíclico permite que el sistema responda dinámicamente ante obstáculos inesperados o cambios en la configuración del espacio navegable.



7.2.2 Detección de obstáculos con LiDAR

- **Objetivo:** Verificar el funcionamiento del sistema de detección de obstáculos mediante datos del sensor LiDAR, con actualización del mapa en tiempo real.
- **Procedimiento:**
 - Se activa el sensor LiDAR en la simulación de Gazebo.
 - El nodo procesa el tópico `/scan` de forma continua.
 - Se monitorizan las alertas generadas cuando se detectan obstáculos cercanos.
 - Se visualiza el resultado en RViz.
- **Evidencias:**

La Figura presenta el sistema de detección en funcionamiento durante el desplazamiento del robot en Gazebo. El sensor LiDAR emite un conjunto de rayos que barren el entorno circundante, generando un patrón radial de mediciones. Los datos del tópico `/scan` contienen arrays de distancias (ranges) para cada ángulo discretizado, permitiendo al nodo de procesamiento construir una representación de los obstáculos cercanos.

En RViz se visualiza la nube de puntos generada a partir de los datos de rango, mostrando la geometría del entorno detectado y permitiendo verificar visualmente la correspondencia entre el espacio físico y las mediciones del sensor. El nodo de detección procesa estos datos y genera alertas cuando la distancia a obstáculos cercanos cae por debajo de umbrales configurables, funcionando como un mecanismo de seguridad para prevenir colisiones. Los obstáculos detectados se representan en el espacio ocupado del mapa, actualizándose en cada ciclo de escaneo para reflejar cambios en la disposición del entorno.

El proceso de detección opera en tiempo real con una frecuencia determinada por la velocidad de actualización del sensor, típicamente entre 10 y 40 Hz. Esta velocidad de actualización permite que el sistema reaccione ante cambios dinámicos en el entorno, como objetos móviles u obstáculos que aparecen durante el desplazamiento del robot. La latencia del procesamiento es crítica para mantener la seguridad operacional, ya que determina cuán rápidamente el sistema puede reaccionar ante nuevos obstáculos.

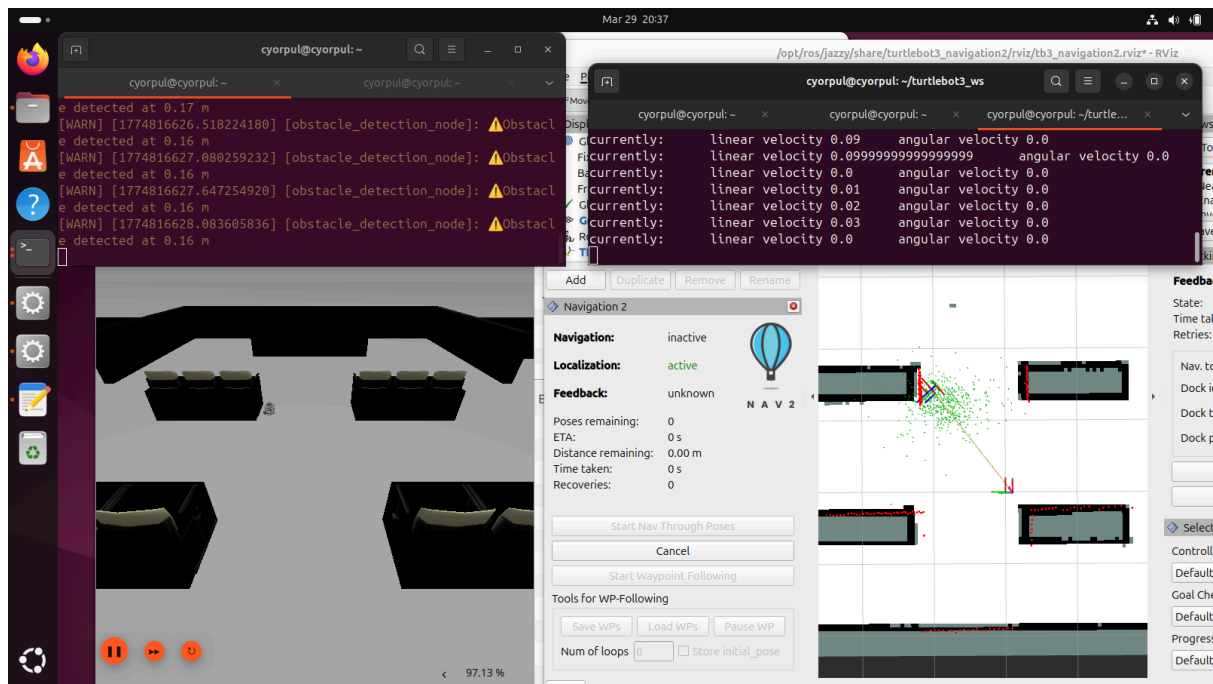


Figura : Obstacle detection y LiDAR

7.2.3 Localización mediante AMCL

Objetivo: Confirmar que el sistema de localización sincroniza correctamente el map server y AMCL, manteniendo la posición exacta del robot mediante corrección de odometría.

Procedimiento:

- Se inicializa el Lifecycle Manager con el map server y el nodo AMCL
- Se desplaza el robot en el entorno simulado con velocidades predefinidas
- Se monitoriza la actualización del tópic `/amcl_pose`
- Se valida la precisión comparando la odometría con la pose corregida

Evidencias:

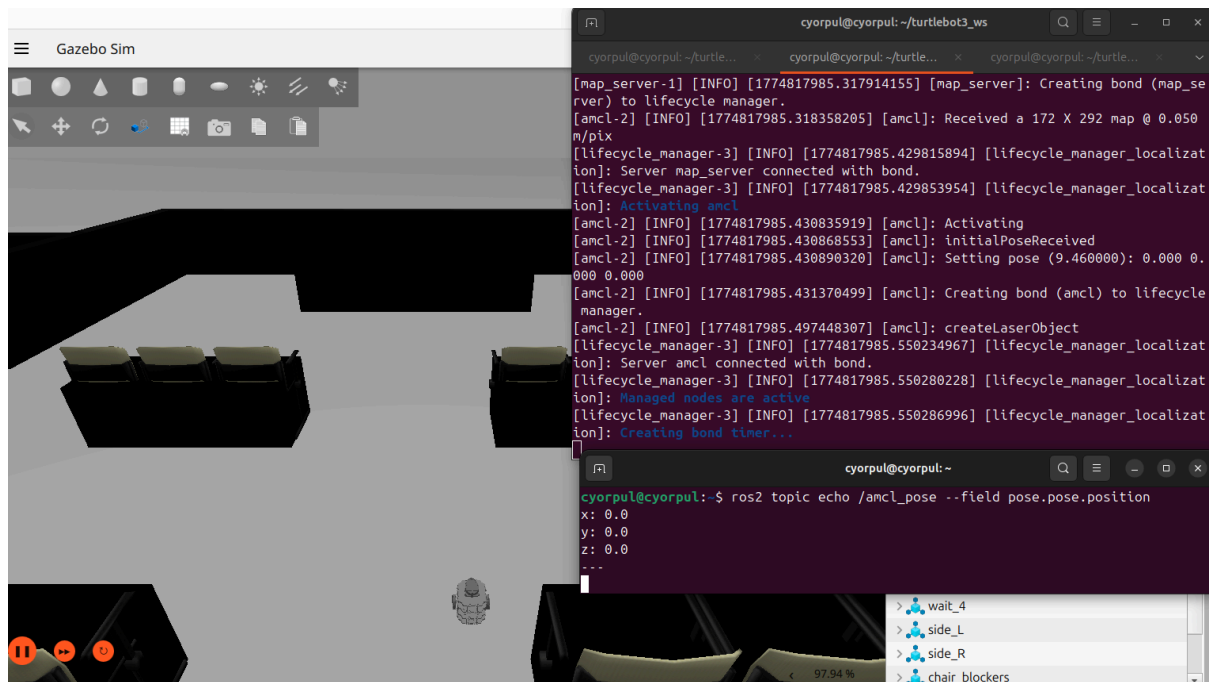
Las Figuras muestran el funcionamiento del sistema de localización en diferentes aspectos. La Figura presenta los logs del Lifecycle Manager durante el proceso de inicialización. El map server carga correctamente el mapa y expone el servicio `'static_map'`, permitiendo que otros nodos accedan a la información espacial del entorno. AMCL se inicializa con parámetros de distribución uniforme o gaussiana según la configuración especificada, generando un conjunto de partículas que cubren el espacio de posibles localizaciones del robot. La sincronización entre nodos se establece mediante el árbol de transformadas (tf tree), que define las relaciones geométricas entre los diferentes sistemas de coordenadas del robot (base del robot, sensor LiDAR, mapa global).

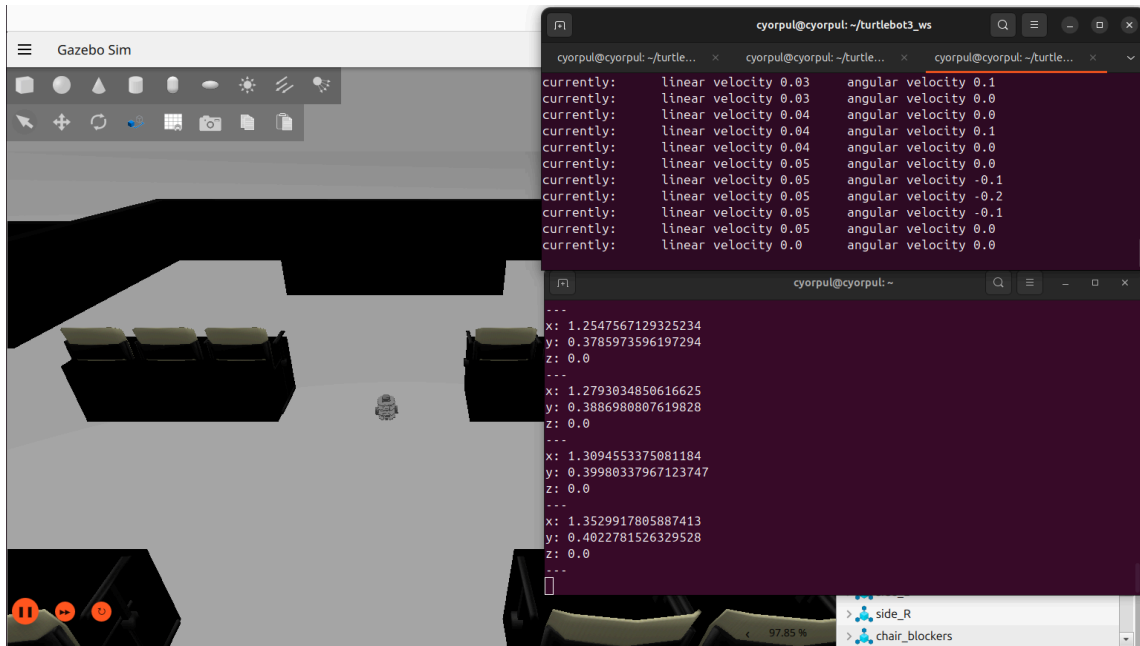
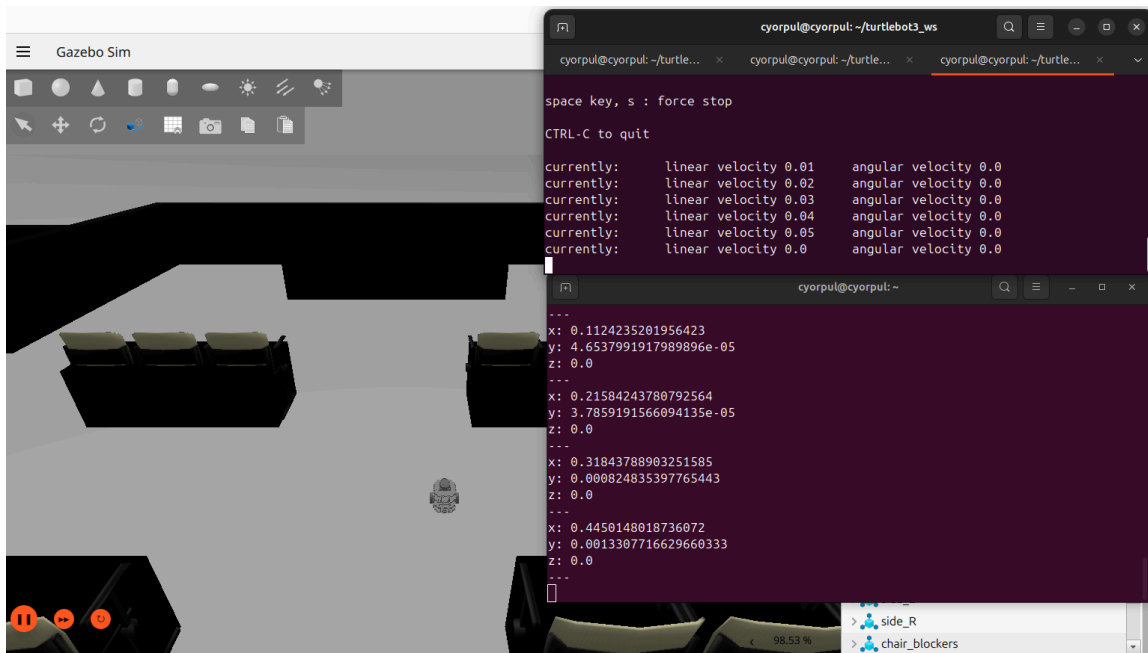
Las Figuras observan el comportamiento de la localización durante el desplazamiento continuo del robot. El robot recibe comandos de velocidad que alteran progresivamente su posición, generando cambios en las coordenadas que se acumulan en la odometría. El sensor LiDAR escanea el entorno de forma periódica y el algoritmo AMCL compara

automáticamente las características observadas (distancias a obstáculos, formas de paredes, ubicación de objetos) con las características esperadas según el mapa precargado.

AMCL ejecuta su ciclo de predicción-corrección en cada iteración: primero predice la nueva pose del robot basándose en el modelo de movimiento y los comandos de velocidad recibidos, luego corrige esta predicción comparando las observaciones reales del LiDAR con el mapa global. El tópico `/amcl_pose` publica las coordenadas (x, y) y orientación (θ) estimadas, junto con la matriz de covarianza que indica el nivel de incertidumbre en cada dimensión de la pose. La corrección mediante LiDAR reduce el error de odometría acumulada que ocurriría si solo se utilizaran los comandos de velocidad, manteniendo la consistencia de la pose dentro del marco de referencia global del mapa.

El algoritmo AMCL funciona mediante una representación probabilística: mantiene múltiples hipótesis de pose, denominadas partículas, que se pesan según la verosimilitud de las observaciones sensoriales. Las partículas con mayor probabilidad de concordancia con las observaciones reciben mayor peso, mientras que las hipótesis menos probables son descartadas o reemplazadas. Este proceso, denominado resampling, permite que el conjunto de partículas converja gradualmente hacia la pose más probable del robot. Esta aproximación probabilística proporciona robustez frente a ruido sensorial y permite que el sistema se recupere de localizaciones erróneas mediante el proceso de corrección continua. La representación probabilística también permite mantener un estimado de la incertidumbre, fundamental para que el sistema de navegación pueda evaluar la confiabilidad de sus estimaciones de pose.





7.2.4 Conexión

Conexión ROS2

Servidor (ws://):

127.0.0.1:9090

Conectar **Desconectar**

Estado: **Conectado**

Conexión ROS2

Servidor (ws://):

127.0.0.1:9090

Conectar **Desconectar**

Estado: **Desconectado**

Figura : Conexion

Objetivo: El propósito principal es transformar una interfaz estática en un sistema de control robusto y profesional capaz de gestionar el ciclo de vida de la comunicación entre la web y el robot.

Procedimiento:

- Preparación del entorno local: se activan simultáneamente el simulador del robot, el puente de comunicación y el servidor de procesamiento de vídeo en la máquina de trabajo.
- Sincronización de la interfaz: se vincula la aplicación web con el servidor local para establecer el canal de intercambio de datos.
- Validación de comunicación: el sistema monitoriza el estado en tiempo real y ofrece feedback visual (Conectado/Desconectado) para confirmar que el canal es seguro.
- Recepción de telemetría y vídeo: una vez establecida la conexión, se inicia la visualización automática de la posición del robot y la transmisión de imágenes en vivo.

- Control activo (Teleoperación): se habilitan los mandos de control para enviar órdenes de movimiento, asegurando que estas solo se procesen si la conexión es estable.
- Gestión de cierre seguro: se permite la desconexión manual o automática, reseteando la interfaz a un estado neutro para evitar el envío de órdenes accidentales.

7.2.5 Control

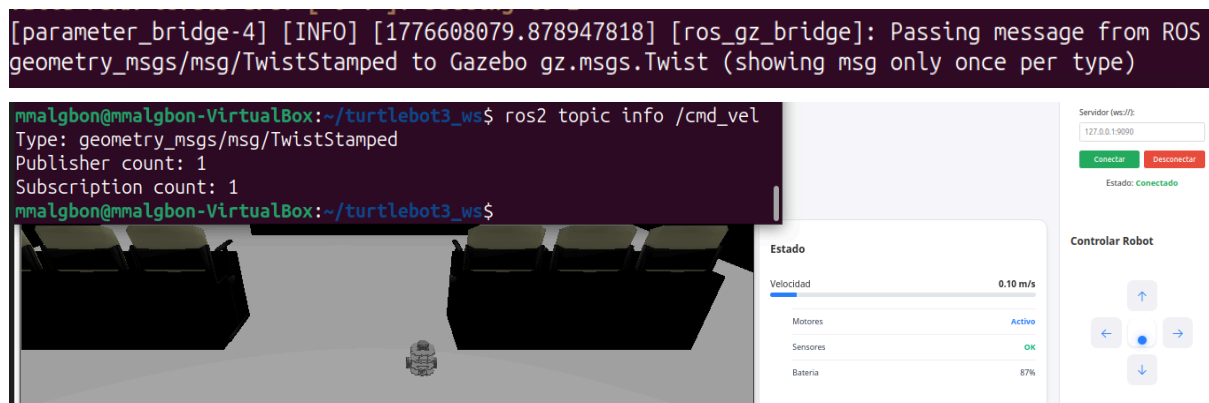


Figura : Control

Objetivo: Confirmar que el control de movimiento se publica desde la web mediante WebSocket, pasa por el bridge ROS-Gazebo y activa el robot con la velocidad indicada.

Procedimiento:

- Se establece la conexión desde la interfaz web con el servidor rosbridge mediante WebSocket.
- Se crea el tópico /cmd_vel desde la web con ROSLIB.Topic, indicando su nombre y el tipo de mensaje.
- Se publica periódicamente un mensaje TwistStamped correctamente estructurado con los valores de velocidad lineal y angular.
- Se selecciona el movimiento deseado y se actualizan las variables internas currentLinear y currentAngular.

Evidencias:

La figura muestra el mensaje del bridge de ROS a Gazebo y el topic info de /cmd_vel, lo que confirma que el mensaje publicado desde la web llegó primero a rosbridge por WebSocket y luego fue reenviado al simulador mediante el bridge. En la consola, el mensaje de inicio de publicación y la información del tópico validan que el sistema está enviando comandos de velocidad de forma periódica y con el tipo esperado TwistStamped.

La lógica del código se basa en un flujo sencillo: primero se conecta la interfaz al WebSocket de ROS, después se crea el publicador del tópico y finalmente se emiten mensajes cada 100 ms mientras la conexión siga activa. Cuando el usuario pulsa un comando, `setMovement()` asigna valores concretos a la velocidad lineal y angular según la dirección elegida, y esos valores son los que se empaquetan en el mensaje publicado.

La barra de velocidad de la interfaz sirve como evidencia visual complementaria, porque se actualiza con el valor absoluto de la velocidad lineal enviada al robot, aunque el movimiento físico no se aprecie claramente en la captura.

7.2.6 Estado

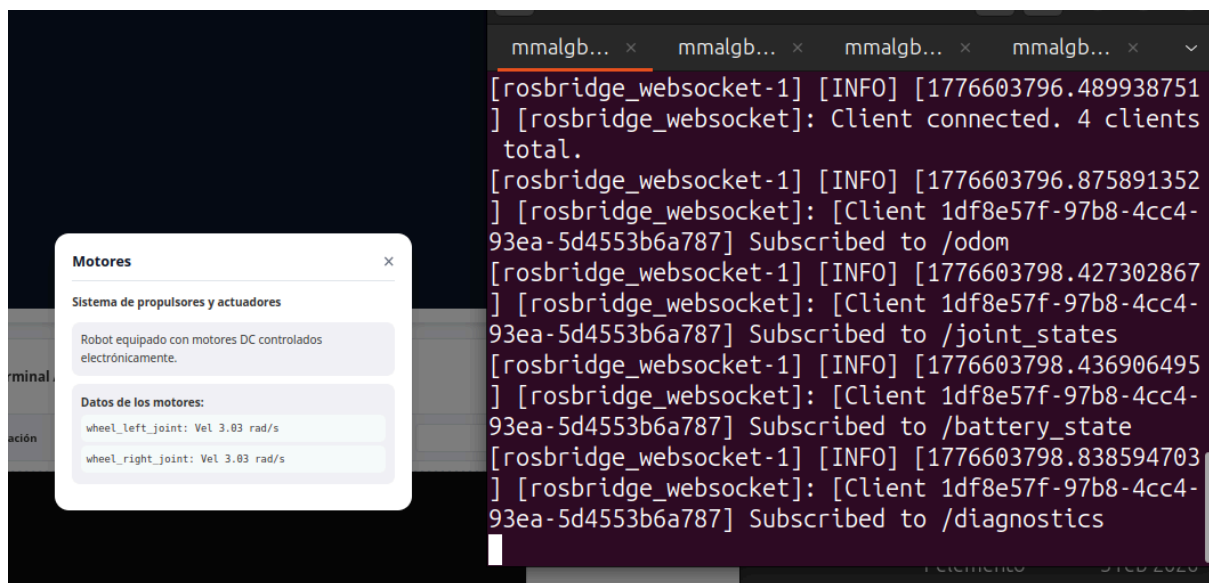


Figura : Estado

Objetivo: Confirmar que la función de estado suscribe correctamente los tópicos del robot y muestra en la tarjeta la información actualizada obtenida desde la consola y los datos recibidos.

Procedimiento:

- Se establecen las suscripciones a los tópicos `/odom`, `/joint_states`, `/battery_state` y `/diagnostics` cuando la conexión ROS está activa.
- Se almacenan los últimos valores recibidos de posición, motores, batería y diagnósticos para mantener el estado actualizado.
- Se abre la tarjeta de información correspondiente desde la interfaz para visualizar los datos del sistema.
- Se verifica en consola que las suscripciones se realizan correctamente mediante los mensajes de confirmación.

Evidencias:

Las evidencias muestran que la función de estado se apoya en la suscripción continua a tópicos de ROS, lo que permite recibir y actualizar datos en tiempo real desde distintos nodos del sistema. En la interfaz, la tarjeta concentra la información más relevante del robot, mientras que la consola confirma que las suscripciones a `/odom`, `/joint_states`, `/battery_state` y `/diagnostics` se activan correctamente.

La primera parte de la evidencia corresponde a la tarjeta de información, donde se visualizan los datos actualizados según la sección seleccionada. En el caso de sensores, se muestra la posición obtenida desde `/odom`, incluyendo coordenadas, orientación y marca de tiempo, lo que permite comprobar que la lectura se actualiza conforme llega nueva información del robot. En motores, batería y diagnóstico ocurre lo mismo: la tarjeta refleja el último estado recibido y traduce esos valores a un formato legible para el usuario, facilitando la supervisión del sistema.

La segunda parte de la evidencia es la consola, donde aparecen los mensajes de confirmación de suscripción. Estos mensajes demuestran que la conexión con ROS está activa y que el sistema escucha correctamente los tópicos definidos, lo cual es esencial para que la tarjeta se mantenga sincronizada con el estado real del robot. Además, este comportamiento valida que la lógica del frontend no solo muestra datos estáticos, sino que responde a eventos en tiempo real generados por los nodos del sistema robótico.

7.2.7 Login

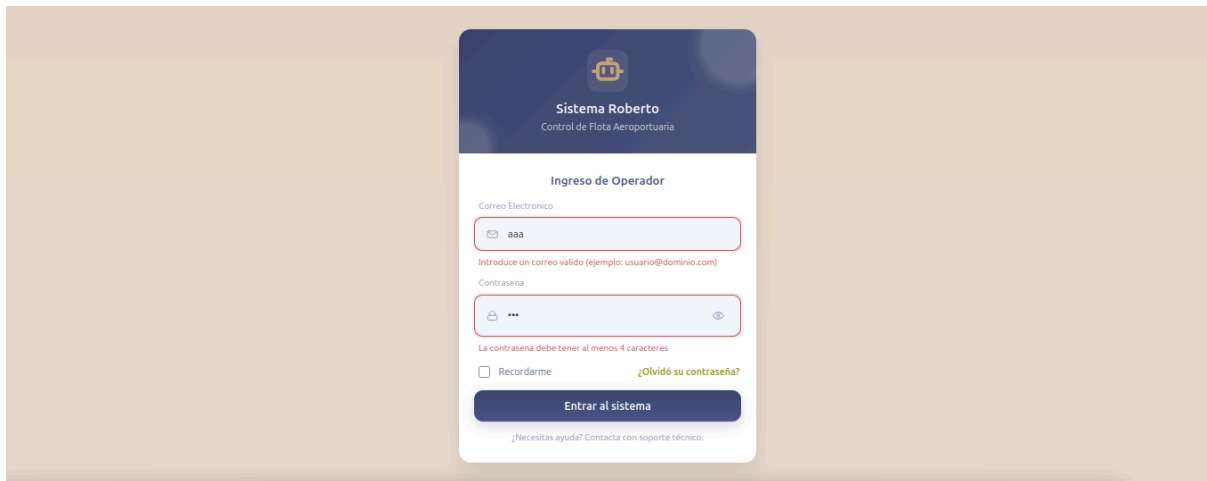
The image shows a mobile application login screen. At the top, there is a dark blue header with a robot icon and the text 'Sistema Roberto' and 'Control de Flota Aeroportuaria'. Below this is a white card titled 'Ingreso de Operador'. The card contains two input fields: 'Correo Electronico' with a placeholder 'aaa' and 'Contraseña' with a placeholder '***'. Below the password field, there is a checkbox for 'Recordarme' and a link for '¿Olvidó su contraseña?'. At the bottom of the card is a blue button labeled 'Entrar al sistema'. A small link at the very bottom says '¿Necesitas ayuda? Contacta con soporte técnico.'

Figura : Login

Objetivo: Confirmar que el sistema de autenticación gestiona correctamente el acceso al dashboard, validando credenciales, protegiendo contraseñas mediante bcrypt y evitando el acceso sin sesión iniciada.

Procedimiento:

- Se inicializa el servidor Express con conexión a la base de datos MySQL
- Se accede al formulario de login desde la interfaz de usuario
- Se valida el estado del formulario en el cliente antes de enviar la solicitud
- Se intenta acceder directamente al dashboard sin autenticación y el sistema redirige al login
- Se envía la petición de autenticación al backend
- Se comprueba la existencia del email en la base de datos
- Se verifica la contraseña utilizando bcrypt
- Se actualiza la contraseña en la base de datos si no está encriptada
- Se crea la sesión o token de autenticación tras credenciales válidas
- Se redirige automáticamente al dashboard tras login correcto
- Se controla el acceso al dashboard mediante middleware de autenticación

Evidencias:

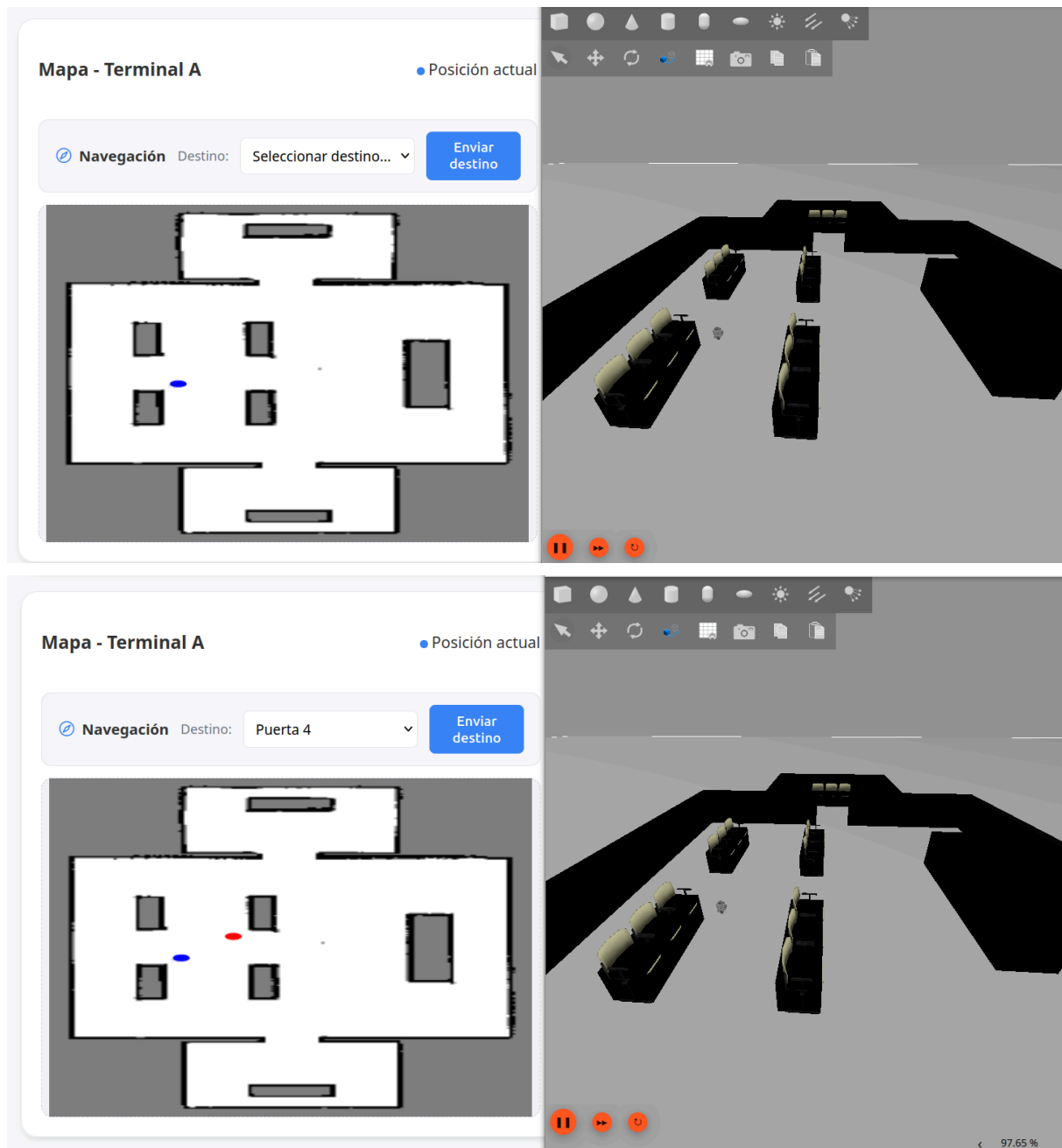
La Figura X presenta el comportamiento de la interfaz de usuario durante el proceso de login. El formulario implementa validaciones en cliente que verifican el formato del email y la presencia de la contraseña antes de permitir el envío. En caso de datos inválidos, la UI muestra mensajes de error , evitando solicitudes innecesarias al servidor y mejorando la experiencia de usuario.

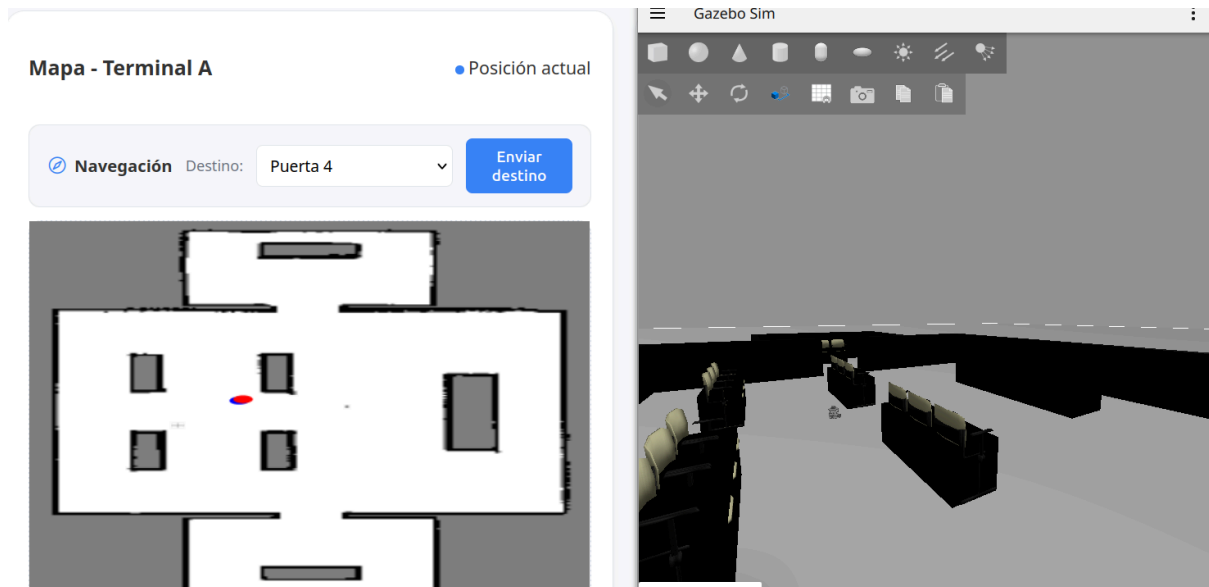
En el backend, el servidor consulta la base de datos para verificar si el email existe y, en caso afirmativo, compara la contraseña mediante bcrypt, garantizando una verificación segura. Si la contraseña no está encriptada, se actualiza automáticamente aplicando hashing y guardándola con una operación UPDATE.

Además, el sistema controla el acceso a rutas protegidas: si un usuario intenta acceder al dashboard sin autenticación, es redirigido al login. Tras un inicio de sesión válido, se genera una sesión o token que permite el acceso, mientras que un middleware verifica la autenticación en cada petición

El sistema en conjunto combina validación en cliente, verificación en servidor y protección criptográfica de credenciales. La utilización de bcrypt introduce un nivel adicional de seguridad mediante hashing y salting, dificultando ataques de fuerza bruta o filtraciones de datos. La arquitectura asegura coherencia entre la interfaz, la lógica de negocio y la persistencia de datos, manteniendo la integridad del proceso de autenticación.

7.2.7 Mapa en Web





Figuras : mapa

Objetivo: Validar que la interfaz web representa correctamente el mapa del entorno, la posición del robot en tiempo real y el envío de objetivos de navegación, asegurando la sincronización entre ROS2, backend y frontend.

Procedimiento:

- Se inicia el sistema completo (ROS2 + backend Express + frontend)
- Se carga el mapa desde ROS mediante ROS2D en la interfaz web
- Se obtiene la posición del robot desde /api/position
- Se actualiza la posición en el mapa de forma continua
- Se selecciona un destino desde el desplegable de zonas
- Se envía la orden de navegación mediante POST /api/sendGoal
- El backend consulta las coordenadas en la base de datos
- Se publica el objetivo en ROS2 (/goal_pose)
- El robot inicia el movimiento en Gazebo (/cmd_vel)

Evidencias:

Las figuras muestran la evolución del sistema durante la ejecución.

En la primera imagen se observa la interfaz inicial, donde el mapa se representa correctamente y la posición actual del robot (punto azul) se muestra en tiempo real, confirmando la correcta recepción de datos desde ROS2.

En la segunda imagen, tras seleccionar un destino ("Puerta 4"), aparece el objetivo marcado en rojo sobre el mapa. Esto valida el flujo completo de envío de metas desde el frontend hasta ROS2.

En la tercera imagen se aprecia el desplazamiento del robot en el entorno de simulación (Gazebo), mientras que en la interfaz web el marcador azul refleja la misma trayectoria en tiempo real, evidenciando la correcta sincronización entre la simulación y la visualización del mapa.

7.2.8 Cámara



Figura : Camara

Objetivo: Validar la correcta integración, transmisión y decodificación en tiempo real del flujo de video capturado por la cámara del robot desde el entorno de simulación (Gazebo) hacia la interfaz web del panel de control técnico.

Procedimiento:

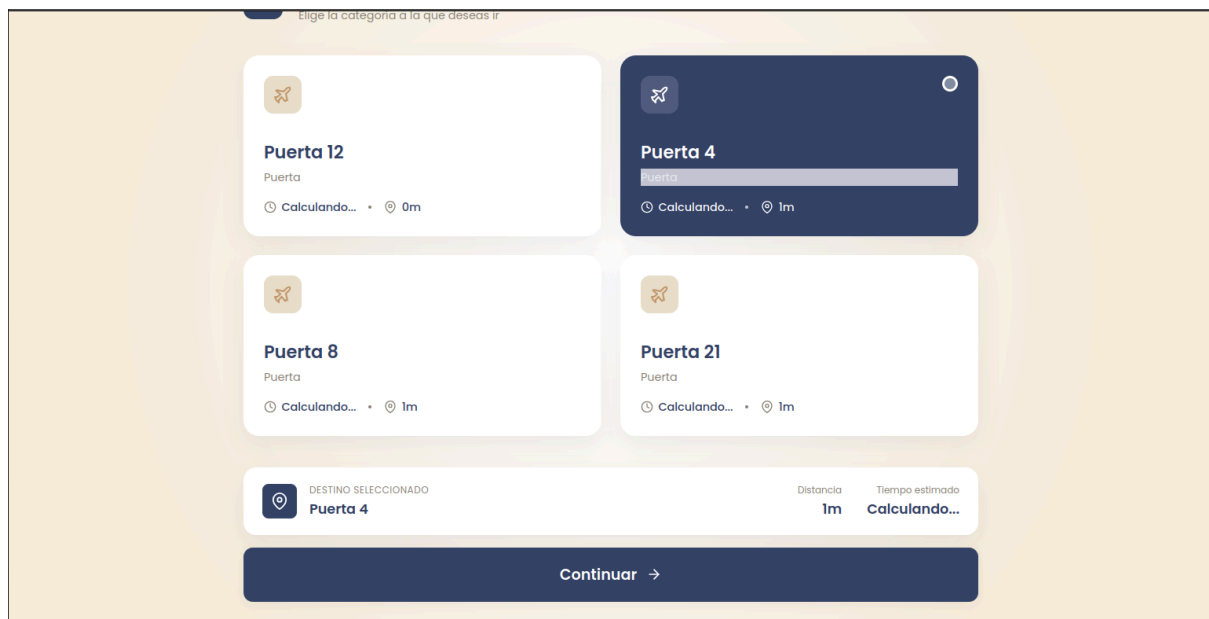
1. Se inicializa el entorno de simulación en Gazebo cargando el modelo del robot equipado con el sensor de cámara.
2. Se ejecuta el nodo puente de imágenes de ROS 2 (`image_bridge`) para exponer el tópico de la cámara (ej. `/camera/image_raw`) y permitir su consumo.
3. Se activa el servidor de comunicaciones WebSocket (`rosbridge_server`) y el backend de Node.js.
4. El operador accede al panel de control (Dashboard Técnico) desde el navegador web.
5. El componente de visualización del frontend se suscribe al tópico de imagen correspondiente a través de `roslibjs`.

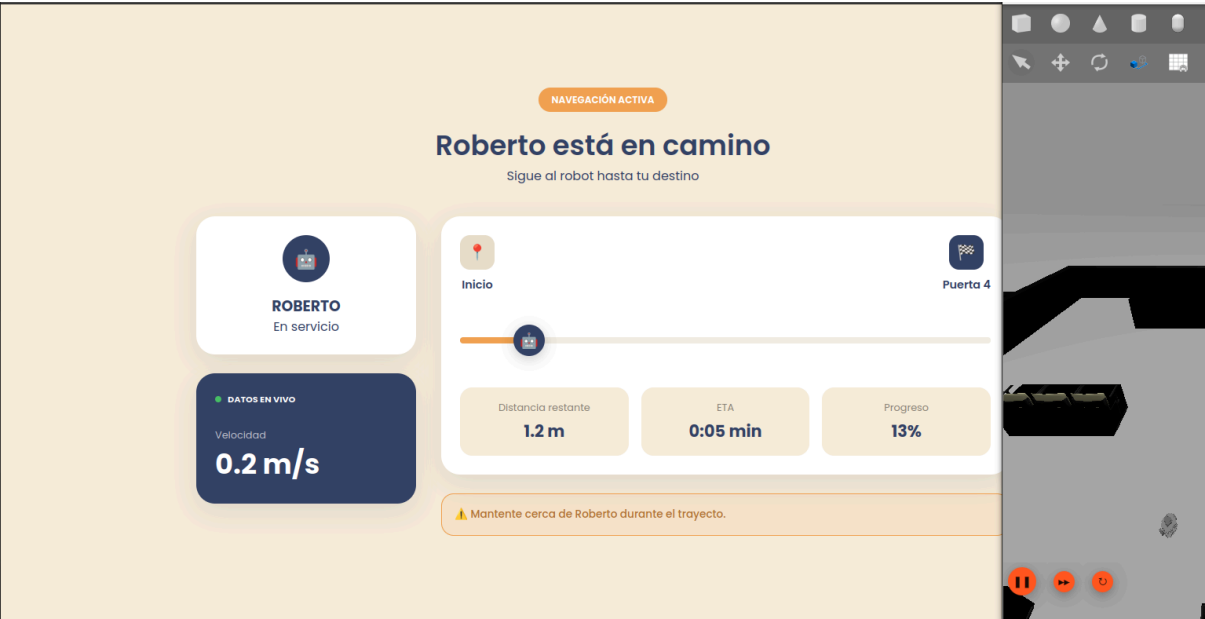
6. La interfaz web procesa los datos en formato Base64 (o comprimido) y renderiza los *frames* continuamente en el lienzo HTML (*canvas* o *img*).

Evidencias: La figura muestra el resultado de la integración del sistema de visión en el panel de control.

- Se observa el componente de monitoreo titulado "Vista de Cámara - Roberto 1" operando con normalidad, acompañado del indicador "EN VIVO" activo, lo que confirma la suscripción y conexión exitosa al flujo de datos de ROS 2.
- El visor reproduce correctamente la perspectiva visual del robot dentro del mundo simulado de Gazebo. Se aprecia la distorsión óptica característica del sensor configurado (lente gran angular u ojo de pez), evidenciando que los paquetes de imagen se están recibiendo, decodificando y dibujando en el navegador de manera íntegra y sincronizada con la simulación.

7.2.9 Interfaz pasajero







Figuras : Interfaz pasajero funcional

Objetivo: Validar el flujo completo de la experiencia del pasajero en la interfaz web, desde la selección interactiva del destino hasta el seguimiento telemétrico del trayecto en tiempo real y la recolección final de métricas de satisfacción, comprobando la sincronización visual con Gazebo y la correcta persistencia de la interacción en la base de datos.

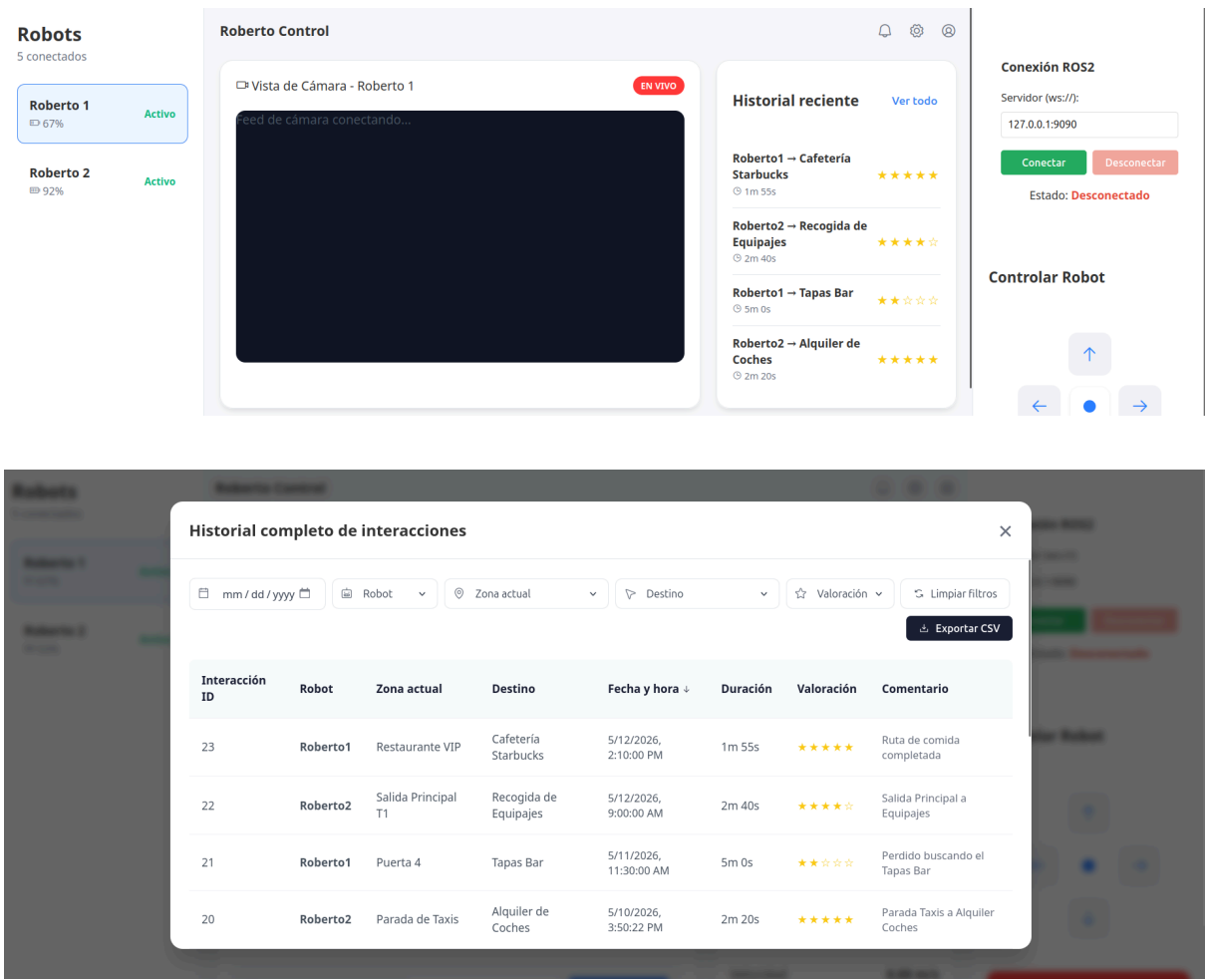
Procedimiento:

1. El pasajero accede a la interfaz gráfica y selecciona un destino específico navegando por las categorías disponibles.
2. Al confirmar la ruta, el frontend envía la orden al backend (`/api/sendGoal`) para iniciar la navegación y redirige automáticamente a la vista de trayecto en curso (`navigation.html`).
3. Durante el movimiento, la interfaz web realiza un *polling* continuo a la API para consumir la telemetría del robot, calculando y renderizando dinámicamente el porcentaje de progreso, la velocidad actual, la distancia métrica restante y el tiempo estimado de llegada (ETA).
4. Al detectar que el robot ha alcanzado el umbral de proximidad al destino, el sistema detiene el progreso de forma autónoma, calcula la duración exacta del viaje y registra la primera fase de la interacción en la base de datos.
5. El sistema redirige automáticamente a la pantalla de llegada (`arrival.html`), inyectando en el DOM los parámetros reales del viaje almacenados en sesión.
6. El pasajero evalúa el servicio mediante un sistema interactivo de estrellas e introduce sus comentarios, actualizando el registro existente en la base de datos (mediante petición `PUT`) para finalizar el ciclo de servicio.

Evidencias: Las figuras muestran la evolución secuencial de la interfaz del pasajero sincronizada con el entorno de simulación:

- En la primera imagen se observa el menú de selección de destinos de la interfaz web, donde el usuario ha marcado correctamente el destino "Puerta 4" y el sistema habilita la acción de inicio de ruta.
- En las imágenes segunda y tercera se demuestra el correcto funcionamiento de la pantalla de "Navegación Activa" operando en paralelo con el simulador Gazebo. Se verifica la actualización en tiempo real de la barra de progreso (avanzando del 13% al 49%), la reducción coherente de la distancia restante (de 1.2 m a 0.7 m) y el reflejo de la velocidad de traslación (0.2 m/s), confirmando el consumo adecuado de la telemetría.
- En la cuarta imagen se presenta la pantalla de valoración tras confirmar la llegada al destino. Se evidencia la inyección exitosa de los datos paramétricos reales del viaje calculados por el sistema (Duración: 1:05, Distancia: 1.4m) en el panel de resumen, así como el despliegue del formulario de *feedback* de 5 estrellas, preparado para enviar la actualización a la base de datos.

7.2.10 Historial



Figuras : Historial

Objetivo:

Implementar un sistema centralizado de monitorización que permita al técnico visualizar, filtrar y exportar el registro histórico de las misiones realizadas por los robots, incluyendo la duración de las trayectorias y el nivel de satisfacción de los usuarios.

Procedimiento:

- Base de Datos: Estructura relacional en MySQL vinculando Interaccion, Robot y Zona mediante Foreign Keys.
- API REST: Endpoints en Express.js con consultas JOIN para convertir IDs técnicos en nombres legibles.
- Lógica Frontend: Motor en JS para paginación de 10 registros y filtrado dinámico (fecha, robot, zona y estrellas).
- Exportación: Función de conversión JSON a CSV para análisis externo en herramientas como Excel.
- Interfaz (UI): Diseño de tarjeta de vista previa en dashboard y modal detallado para auditorías.

Evidencias:

- Captura 1 (Dashboard): Muestra el componente "Historial Reciente" integrado en el panel de control, facilitando la visualización de las últimas 4 interacciones y sus puntuaciones de estrellas sin cambiar de pestaña.
- Captura 2 (Modal de Historial): Muestra la tabla completa con todas las herramientas funcionales:
 - Selector de Fecha: Para auditorías de días específicos.
 - Dropdowns Dinámicos: Filtros de Robot, Zona Actual y Destino poblados directamente desde la base de datos.
 - Paginación: Navegación fluida entre los registros de prueba actuales.
 - Botón de Exportar: Funcionalidad para descarga de datos en formato .csv.

7.2.11 Detección de Personas (OpenCV)

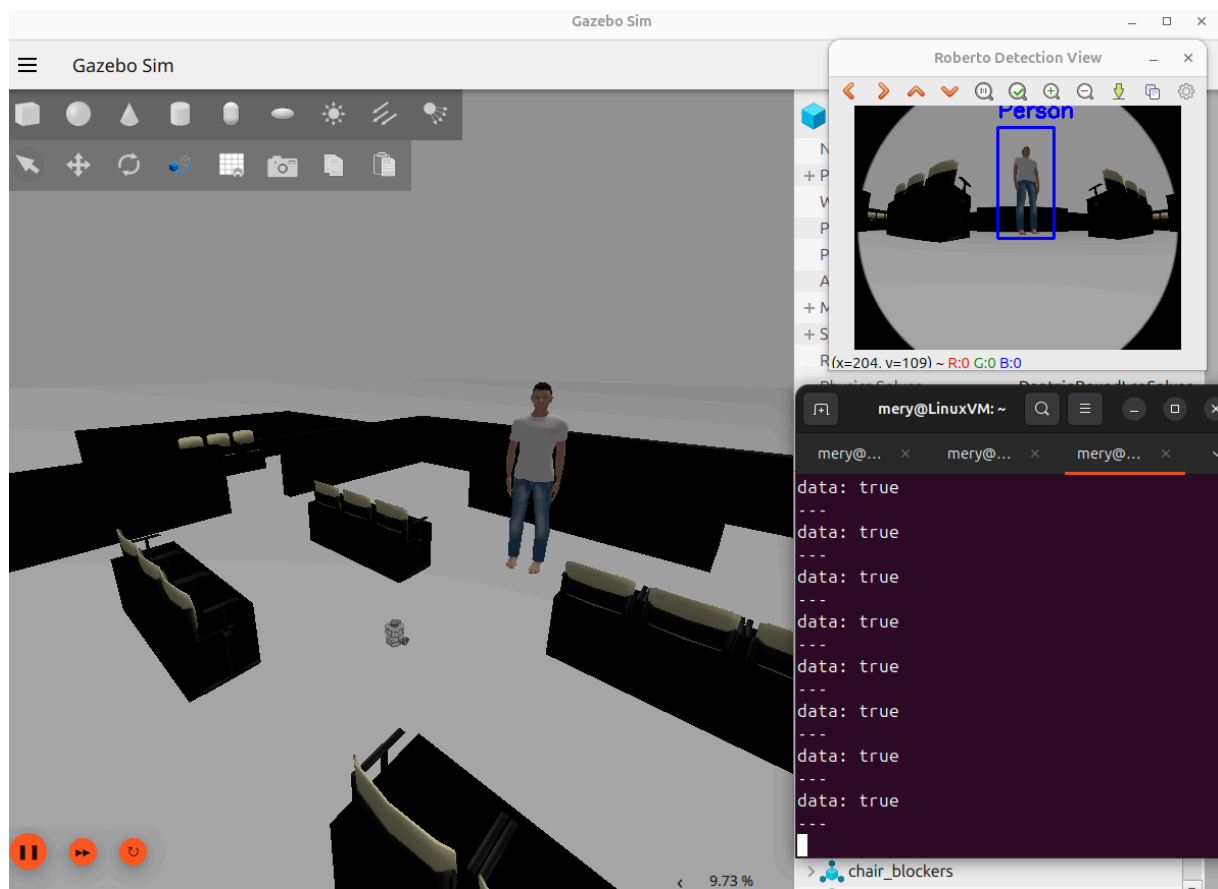


Figura : Deteccion de Personas OpenCV

Objetivo:

Validar que el robot Roberto identifica correctamente a los pasajeros en el entorno aeroportuario mediante visión artificial y comunica este estado al resto del sistema a través de ROS2.

Procedimiento:

- Se inicializa el sistema en Gazebo utilizando el robot con cámara (burger_cam) en el mapa del aeropuerto.
- Se coloca un modelo humano frente al robot para simular la presencia de un usuario que requiere asistencia.
- Se ejecuta el nodo de detección basado en la librería OpenCV, el cual analiza el flujo de vídeo en tiempo real.
- Se utiliza el clasificador Haar Cascade configurado para detectar cuerpos completos (haarcascade_fullbody).
- El nodo procesa los frames y, al confirmar la detección, publica un mensaje booleano en el tópico /person_detected.

Evidencias:

La figura muestra la integración total del módulo de percepción en tres niveles:

- Entorno: Roberto frente al pasajero en el área de espera (Gazebo).
- Visión: Reconocimiento visual del humano mediante un recuadro azul y la etiqueta "Person" (OpenCV).
- Estado: Publicación constante del valor data: true en el tópico /person_detected, permitiendo al robot tomar decisiones de interacción (Terminal).

7.2.13 Detección de gestos (OpenCV)

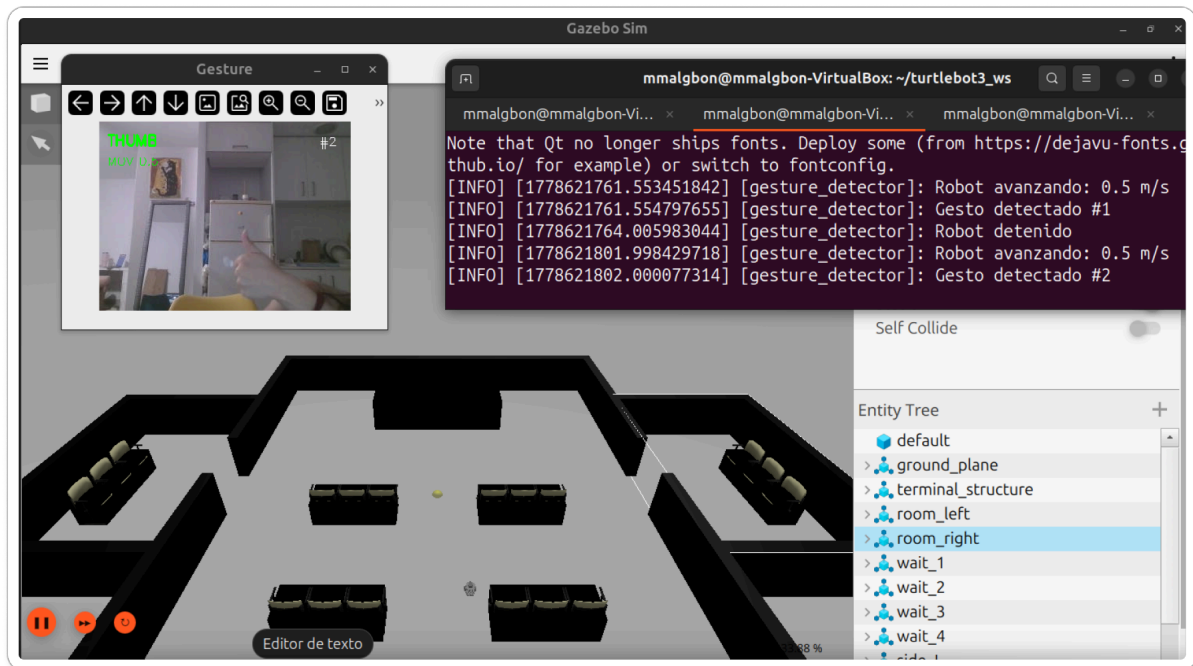


Figura : Deteccion de gestos OpenCV

Objetivo:

Validar que el robot Roberto responde al gesto de "llamada" realizado por un pasajero, detectando el pulgar extendido con el resto de dedos cerrados (puño) y publicando comandos de navegación a través de ROS2.

Procedimiento:

- Se inicializa el sistema en Gazebo utilizando el robot con cámara (burger_cam) en el mapa del aeropuerto.
- Un pasajero simula una llamada al robot mostrando la mano con el pulgar extendido y el resto de dedos cerrados (puño).
- Se ejecuta el nodo de detección basado en la librería MediaPipe HandLandmarker, el cual analiza el flujo de vídeo en tiempo real.
- El nodo detecta los 21 puntos de referencia (landmarks) de la mano y calcula las distancias entre la punta del pulgar y su base, así como entre la punta y base de los dedos índice, medio, anular y meñique.

- Al confirmar el gesto (pulgar extendido Y resto de dedos cerrados), se publica un comando TwistStamped en el tópic `/cmd_vel` con velocidad configurable (`approach_speed`), haciendo que el robot avance hacia el pasajero.
- Cuando el pasajero cierra completamente el puño o retira la mano del campo de visión, el robot se detiene automáticamente.

Evidencias:

La figura muestra la integración total del módulo de percepción en tres niveles:

- Entorno: Roberto frente al pasajero realizando el gesto de llamado en el área de espera (Gazebo).
- Visión: Reconocimiento de la mano con puntos de referencia verdes y el texto "THUMB" en pantalla (OpenCV/MediaPipe).
- Estado: Publicación del comando de velocidad en `/cmd_vel` con valor `linear.x = 0.5` m/s en el tópic TwistStamped, permitiendo al robot acercarse al usuario (Terminal).

7.3 Conclusión de las pruebas

Las pruebas realizadas demuestran el funcionamiento integrado del sistema de navegación autónoma como un ciclo continuo de percepción, estimación y acción. El sensor LiDAR genera datos de escaneo en cada ciclo, que alimentan simultáneamente dos procesos: la detección de obstáculos que actualiza el mapa de ocupación dinámico, y el algoritmo de localización AMCL que corrige la estimación de pose del robot. El costmap se actualiza en tiempo real con la información de ocupación proporcionada por estos sensores, representando zonas de navegación libre y zonas de colisión potencial. El planificador de rutas global calcula trayectorias considerando este mapa actualizado, generando waypoints que el controlador local ejecuta mediante comandos de velocidad. A este flujo se añade el procesamiento de imagen con OpenCV, permitiendo al robot identificar pasajeros en tiempo real para una interacción más inteligente.

Este flujo cíclico permite que el robot mantenga una estimación continua y actualizada de su estado y del entorno que lo rodea, con latencias de procesamiento que permiten respuestas reactivas ante cambios inesperados. La arquitectura del sistema demuestra cómo cada componente se integra con los demás: la información de localización permite referenciar la detección de obstáculos al mapa global, la visión artificial permite distinguir entidades humanas, y el conocimiento del entorno permite planificar trayectorias seguras y eficientes.

8. Funcionamiento del grupo

Actas

Fecha	Puntos tratados	Conclusión
02/03	Definición de tareas, reparto de trabajo por módulos (simulación, localización, navegación)	Sprint bien planificado
06/03	Problemas de integración, redistribución de tareas para equilibrar carga	Inicio lento
10/03	Avances en entorno, cada miembro continúa con su módulo asignado, resolución de dudas	Se gana ritmo
14/03	Varias tareas completadas, ajuste de tareas pendientes según progreso individual	Buen progreso
16/03	Revisión rápida, trabajo individual sin cambios en asignación	Actividad reducida
18/03	Sin avances significativos, sin redistribución debido a baja disponibilidad	Baja productividad
20/03	Retoma del trabajo, reorganización de tareas críticas entre miembros	Recuperación del ritmo
22/03	Progreso constante, cada miembro avanza en su parte asignada	Sprint estable
24/03	Validación de funcionalidades, colaboración en tareas finales	Fase final
26/03	Ajustes finales, apoyo entre miembros para cerrar tareas pendientes	Listos para cierre
30/03	Evaluación del sprint, análisis de problemas y organización del trabajo	Sprint completado

Fecha	Puntos tratados	Conclusión	Participantes y acciones
01/04/2026	Definición del alcance del sprint y reparto de módulos principales.	Se acordó dividir el trabajo por bloques funcionales para avanzar en paralelo sin depender del trabajo diario conjunto.	Maria: asumió T01, T02 y T03, centrando su trabajo en el acceso técnico y el control del robot. Meryame: asumió T04, T08 y T10, orientando su trabajo a navegación, destino y cierre del trayecto. Christopher: asumió T05, T06 y T07, enfocándose en cámara, conexión e interfaz inicial del pasajero.
05/04/2026	Estado del robot, control manual y primer ajuste de interfaz.	Se confirmó que la parte técnica del operador debía quedar estable antes de continuar con módulos de navegación y visualización.	Maria: dejó avanzadas las pantallas y la lógica base del login, además de la estructura de estado y comandos. Meryame: revisó la futura integración de destino y llegada para mantener coherencia con el flujo del viaje. Christopher: preparó la parte visual de conexión y cámara para integrarla más adelante con el resto del sistema.
09/04/2026	Recepción e integración de la base de datos del sistema.	La base de datos quedó incorporada como soporte general del proyecto y permitió continuar con el desarrollo modular.	Maria: adaptó sus pantallas y flujos para consultar y enviar datos al modelo definitivo. Meryame: alineó sus módulos de destino y valoración con la estructura real de almacenamiento. Christopher: revisó la compatibilidad de los módulos visuales y de conexión con la nueva base de datos.
12/04/2026	Selección de destino y preparación del flujo de navegación.	Se validó que el usuario debía poder elegir destino de forma simple y que la navegación continuara de manera automática.	Maria: dejó cerrada la parte de inicio y acceso técnico, manteniéndola como funcionalidad secundaria del sistema. Meryame: desarrolló la selección de destino y la lógica de llegada para el usuario final. Christopher: descubrió que el robot del mundo no llevaba cámara.

Fecha	Puntos tratados	Conclusión	Participantes y acciones
16/04/2026	Cámara, conexión del robot y estado general del sistema.	Bloqueo: Inestabilidad persistente en la conexión del servidor. Se decidió trabajar en local como solución temporal.	Maria: terminó la parte de login y control del operador, dejando el acceso técnico operativo. Meryame: consolidó destino, navegación e integración del mapa. Christopher: trabajó en cámara y conexión, afectadas por los fallos.
19/04/2026	Repaso final de los módulos desarrollados y validación de entregables.	El sprint cerró con el trabajo dividido por partes funcionales.	Maria: cerró T01, T02 y T03 con el flujo de operador y control del robot preparado para validación. Meryame: cerró T04, T08 y T10 con destino, navegación y valoración integrados. Christopher: cerró T05, T06 y T07 con cámara, conexión e interfaz inicial funcionando como base del sistema.

Fecha	Puntos tratados	Conclusión	Participantes y acciones
27/04/2026	Se revisó el progreso en gestos, detección de personas y monitorización del pasajero.	Algunos gestos no se detectaban correctamente; se planificaron ajustes para esta semana.	Maria: ajustó OpenCV para gestos y revisó pequeños detalles de la web del pasajero. Meryame: mejoró detección de personas y actualizó HTML. Christopher: verificó que todo se vea correcto en la web.
29/04/2026	Probamos cómo funcionan juntos gestos, personas y monitorización en la web ya existente.	Se identificaron retrasos en la actualización de datos.	Maria: ajustó gestos y verificó botones de la web. Meryame: mejoró detección de personas y modificó estilos CSS simples. Christopher: revisó seguimiento pasajeros y ajustó algunas secciones de la web con Bootstrap.
02/05/2026	Buscamos mejorar rendimiento de gestos y personas y que la web cargue rápido.	Todo funciona más fluido.	Maria: optimizó gestos con OpenCV. Meryame: aceleró la detección de personas. Christopher: ajustó pequeños detalles de la web y botones Bootstrap.

Fecha	Puntos tratados	Conclusión	Participantes y acciones
05/05/2026	Analizamos qué salió bien y qué falta antes de la demo.	Decidimos documentar los cambios y hacer pruebas finales de integración.	Maria: revisó algunos estilos web. Meryame: documentó detección de personas y revisó comentarios HTML. Christopher: documentó seguimiento pasajeros y revisó que la web funcione bien en distintos tamaños de pantalla.
09/05/2026	Ajustes de última hora en gestos, personas y monitorización antes de la demo.	Todo listo para la demo, solo detalles menores de interfaz.	Maria: revisó navegación web. Meryame: revisó CSS. Christopher: revisó cámara y algunos componentes Bootstrap.
13/05/2026	Presentación final de gestos, personas y monitorización en la web del pasajero.	Demo exitosa; se sugirieron pequeños ajustes visuales para futuras iteraciones.	Maria: supervisó documentación. Meryame: supervisó personas y detección visual. Christopher: supervisó cámara y revisó diseño web general.

Burndown

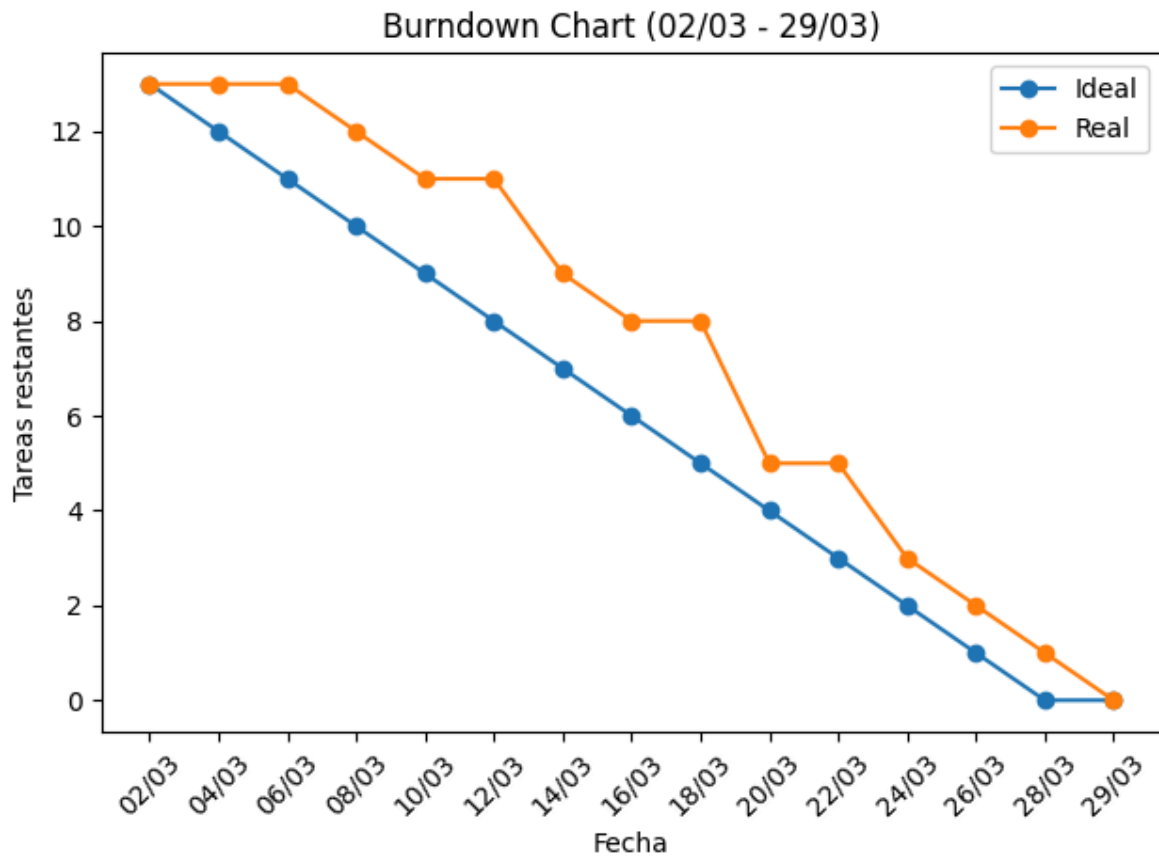


Figura : Burndown Sprint 1

El burndown chart refleja la evolución del trabajo a lo largo del sprint, mostrando un progreso no lineal condicionado por un inicio lento, un periodo de menor actividad durante las fechas festivas y una posterior recuperación del ritmo. A medida que avanzó el sprint, el equipo logró estabilizar el flujo de trabajo y completar las tareas de forma progresiva, alcanzando los objetivos dentro del plazo establecido

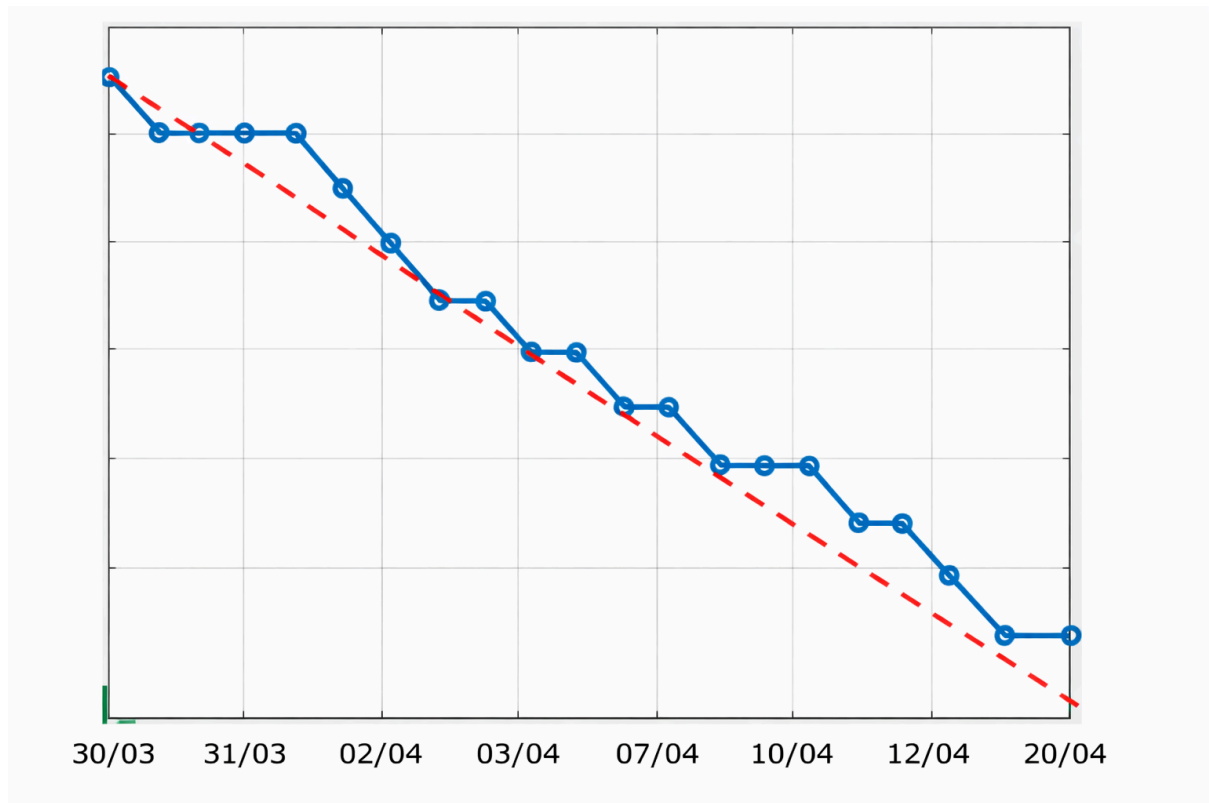


Figura : Burndown Sprint 2

Al observar el gráfico:

- En algunos puntos, el progreso real se mantiene cercano al ideal, lo que indica que el trabajo se completó de manera eficiente.
- Sin embargo, también hay momentos en los que el progreso real se desvía de la línea ideal, lo que sugiere que hubo días con menor avance en comparación con lo planeado.

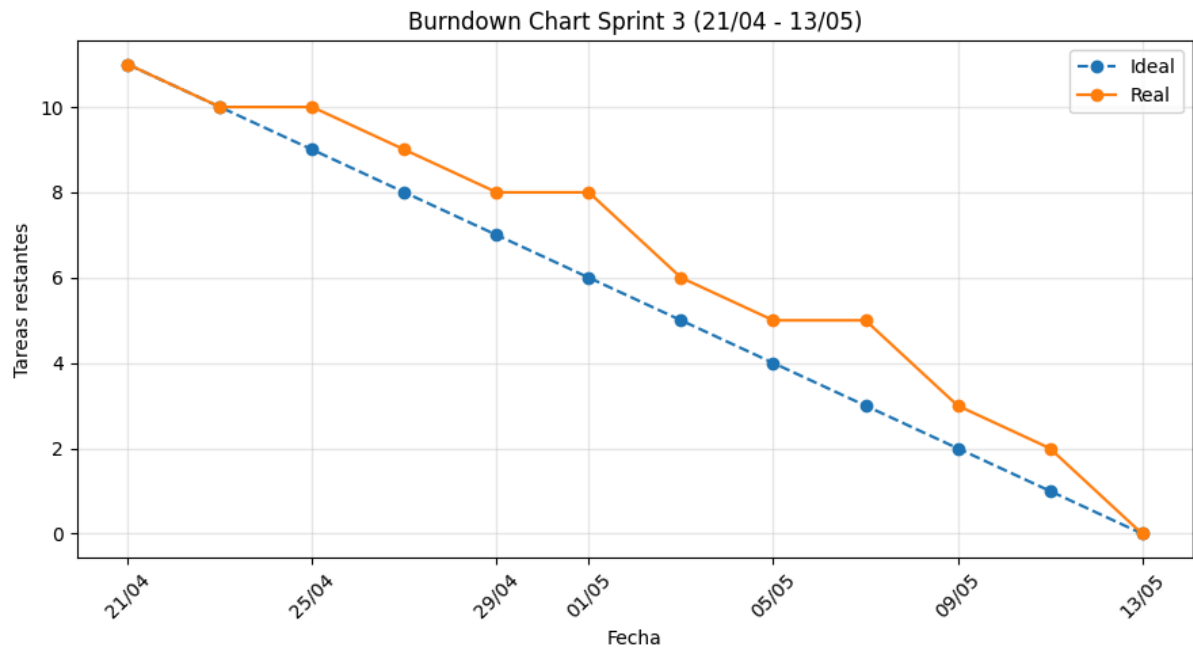


Figura : Burndown Sprint 3

El burndown chart del Sprint 3 refleja un avance continuo y cercano a la planificación inicial. Aunque durante algunas fases hubo pequeñas fluctuaciones en el ritmo de trabajo debido a tareas de integración, pruebas y ajustes del sistema, el equipo mantuvo una evolución estable a lo largo del sprint. Finalmente, todas las tareas previstas fueron completadas dentro del plazo establecido, manteniendo un flujo de trabajo eficiente y organizado.

Anexo 1. Tabla de historial de versiones

Versión	Autor	Descripción	Fecha
v1.0	María	Versión inicial del documento, estructura	4/03
v1.1	María Christopher Meryame	Introducción, contexto y objetivos Mockup Diagramas y arquitectura	11/03
v1.2	Meryame	Diseño y arquitectura	18/03
v1.3	Christopher	Definición requisitos	22/03
v1.4	Maria	Base de datos	25/03
v1.5	María Christopher	Funcionamiento del grupo (actas y burndown)	16/03

	Meryame		
v1.6	Meryame	Corrección diagramas	6/04
v1.7	Maria	Detallar componentes e implementación, añadir más información técnica	10/04
v1.8	Christopher	Formatear las pruebas de las funcionalidades	19/04
v1.9	María Christopher Meryame	Actualizar figuras	24/04
v1.10	Meryame	Corrección arquitectura	5/05
v1.11	Christopher	Actualizar mockups	7/05
v1.12	Maria	Prueba funcionslidad	10/05
v1.13	Meryame	Describir partes de OpenCV	12/05